WhatNext Vision Motors: Shaping the Future of Mobility with Innovation and Excellence

User Story:

WhatsNext Vision Motors is revolutionizing its customer experience and operational efficiency with a cutting-edge Salesforce CRM implementation. The project streamlines the vehicle ordering process by auto-assigning orders to the nearest dealer based on customer location and preventing orders for out-of-stock vehicles. Automated workflows update order statuses dynamically and send scheduled email reminders for test drives. Key technical implementations include Apex triggers for stock validation, batch jobs for stock updates, and scheduled Apex for automated order processing. This initiative enhances customer satisfaction, improves order accuracy, and boosts overall operational efficiency.

1. INTRODUCTION

1.1 Project Overview :

WhatNext Vision Motors implemented a Salesforce-based CRM solution to modernize vehicle ordering and dealership operations. The system automates vehicle order placement, validates stock availability, assigns orders to the nearest dealer, schedules test drive reminders, and updates order statuses dynamically. This transformation replaces manual and disconnected processes with a centralized, automated platform, improving operational efficiency and customer satisfaction.

1.2 Purpose :

The purpose of this project is to automate vehicle ordering, stock management, dealer assignment, and customer engagement using Salesforce CRM. The system aims to reduce manual intervention, eliminate order errors, provide real-time tracking, and enhance overall business productivity while delivering a seamless customer experience.

2. IDEATION PHASE

2.1 Problem Statement (Customer Problem Statement Template)

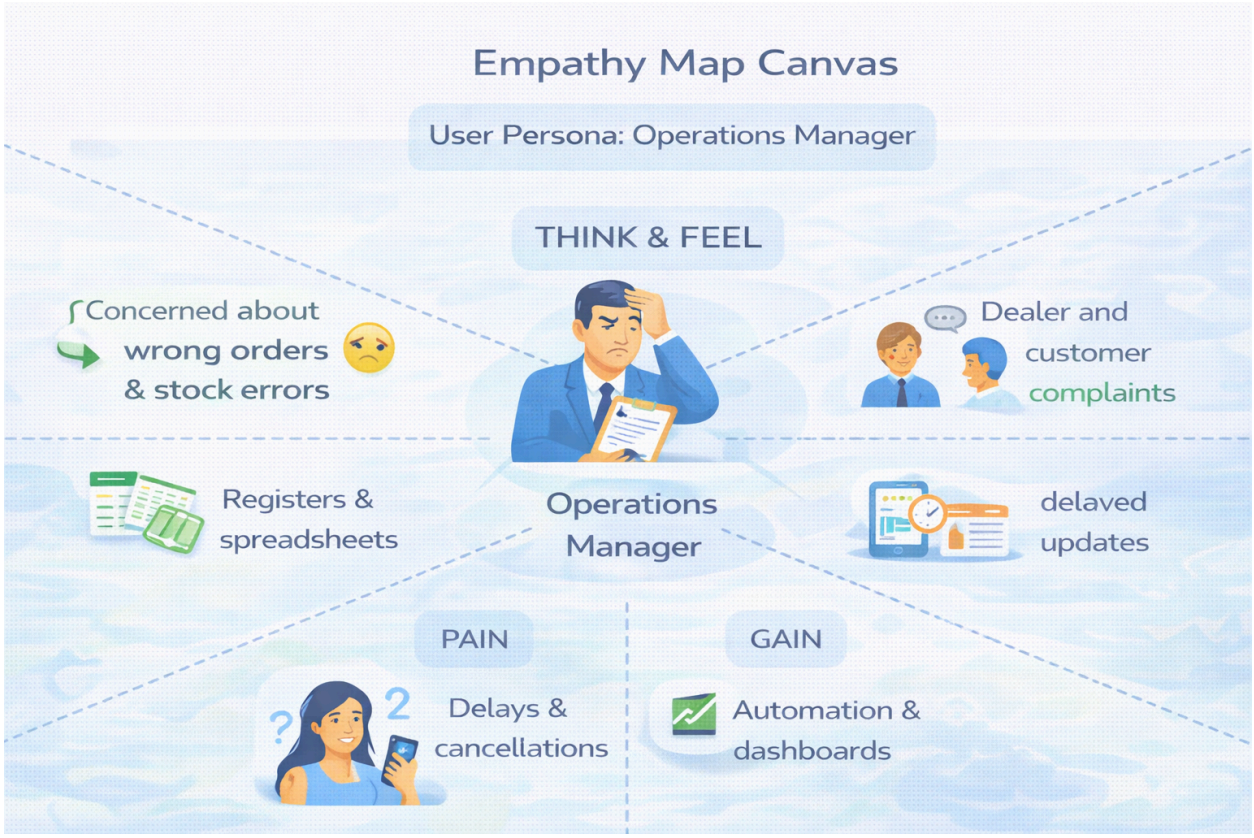
PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	Operations Manager	manage vehicle orders	stock info is manual	systems are disconnected	stressed
PS-2	Customer	place vehicle orders	availability unclear	delayed updates	uncertain

PS	I am	I'm trying to	But	Because	Which makes me feel
PS-1	Operations Manager	manage vehicle orders	stock info is manual	systems are disconnected	stressed
PS-2	Customer	place vehicle orders	availability unclear	delayed updates	uncertain

2.2 Empathy Map Canvas

User Persona: Operations Manager

- **Think & Feel:** Concerned about wrong orders & stock errors
- **Hear:** Dealer and customer complaints
- **See:** Registers & spreadsheets
- **Say & Do:** Manual follow-ups
- **Pain:** Delays & cancellations
- **Gain:** Automation & dashboards



2.3 Brainstorming & Idea Prioritization

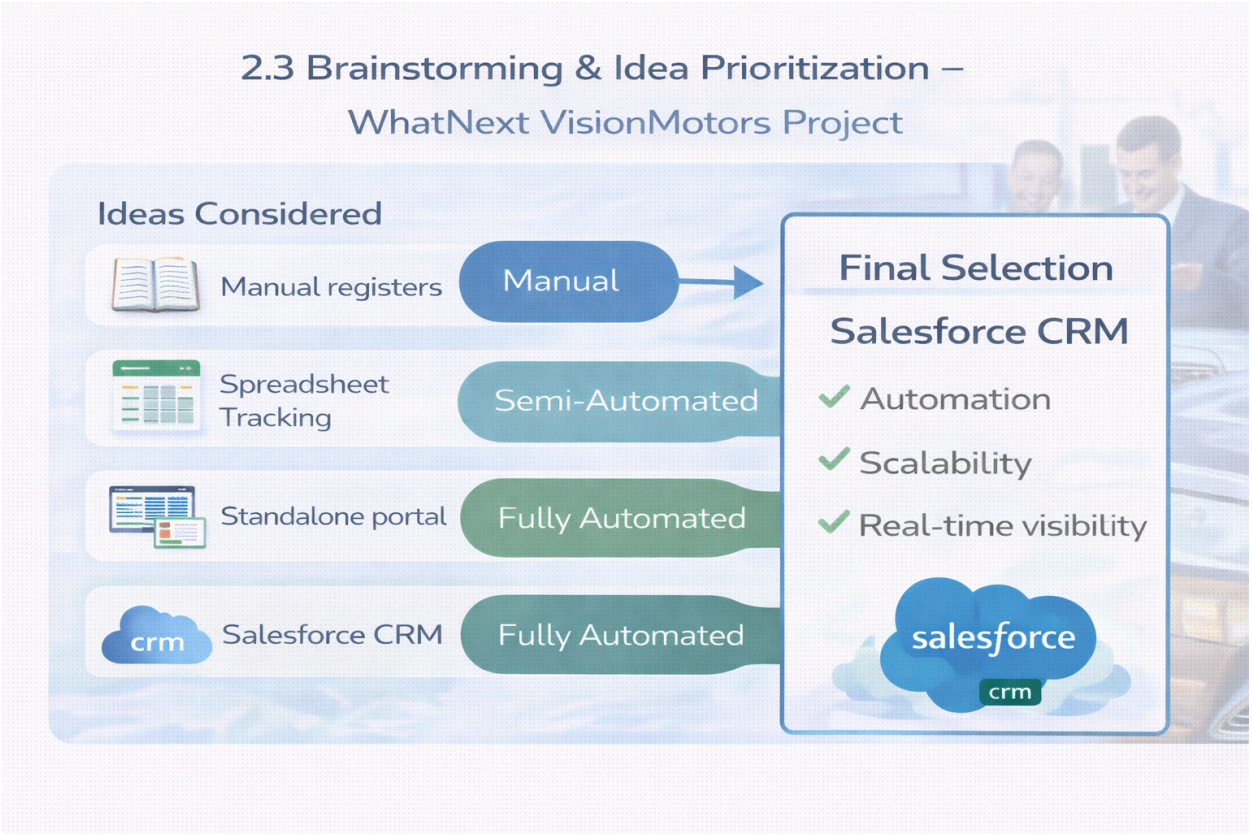
Ideas Considered

- Manual registers
- Spreadsheet tracking
- Standalone portal
- Salesforce CRM

Final Selection

Salesforce CRM selected due to:

- Automation
- Scalability
- Real-time visibility



3.REQUIREMENT ANALYSIS PHASE

3.1 Customer Journey Map

Stages:

- Inquiry → Vehicle Selection → Order Placement → Dealer Assignment → Stock Validation → Test Drive → Confirmation → Reporting

3.2 Solution Requirements

Functional Requirements

FR No	Requirement
FR-1	Vehicle management
FR-2	Dealer management
FR-3	Customer order creation
FR-4	Stock validation
FR-5	Dealer auto-assignment

FR-6	Test drive scheduling
FR-7	Reports & dashboards

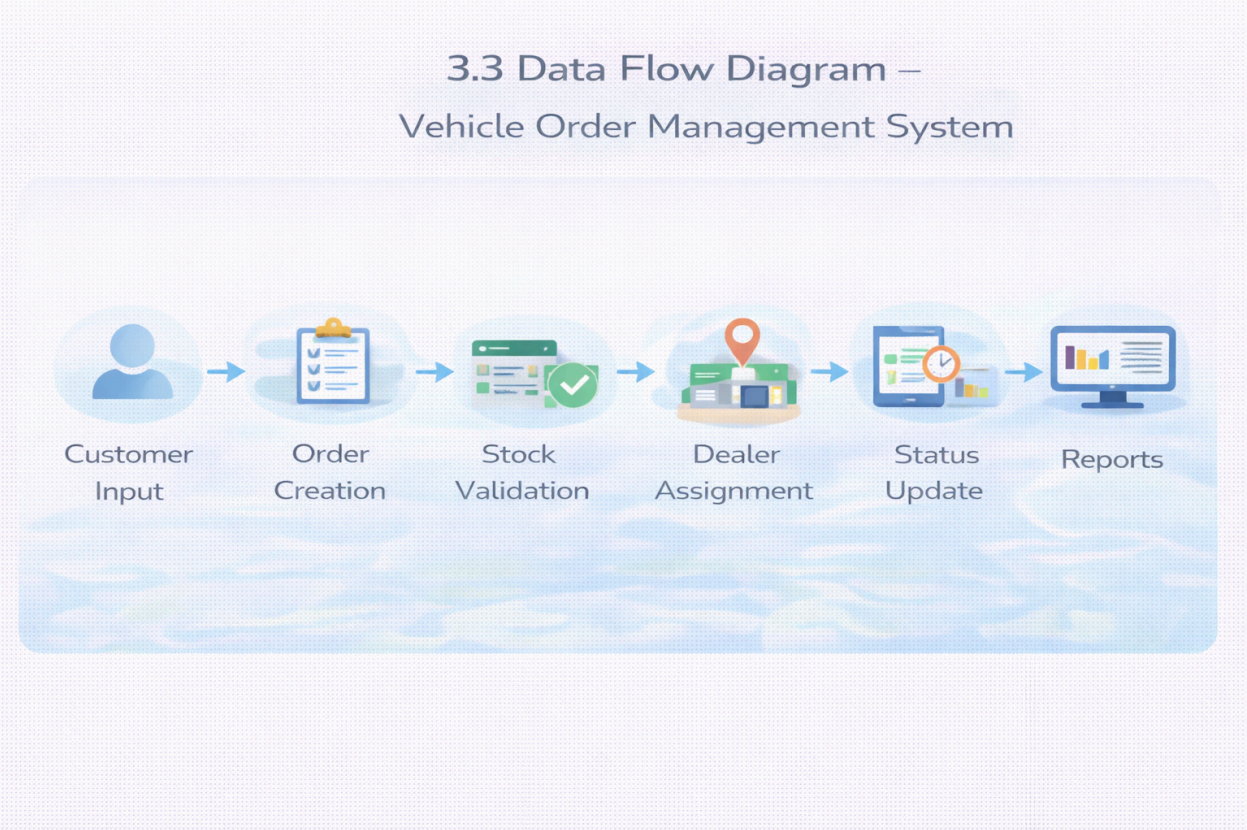
Non-Functional Requirements

NFR	**Description**
Usability	Lightning UI for easy navigation
Security	Role-based access control
Reliability	Accurate automation
Performance	Fast order processing
Availability	24/7 cloud access
Scalability	Multi-dealer support

3.3 Data Flow Diagram

Flow:

Customer Input → Order Creation → Stock Validation → Dealer Assignment → Status Update → Reports



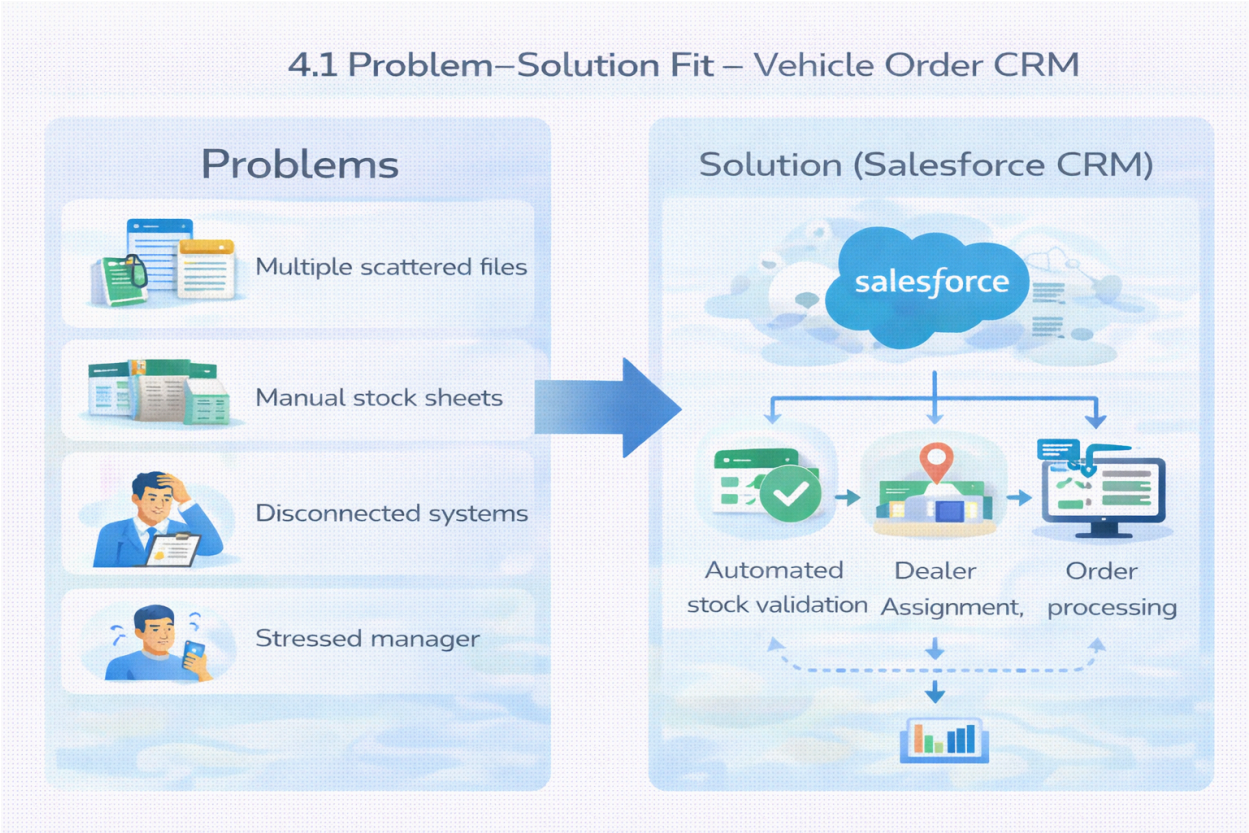
3.4 Technology Stack

Layer	**Technology**
UI	Salesforce Lightning
Logic	Salesforce Flows
Database	Salesforce Objects
Automation	Validation Rules, Triggers
Processing	Batch Apex, Scheduled Apex
Reporting	Reports & Dashboards

4. PROJECT DESIGN PHASE

4.1 Problem–Solution Fit

The system eliminates fragmented operations by integrating automated stock validation, dealer assignment, and order processing within Salesforce.



4.2 Proposed Solution

Functional Requirements

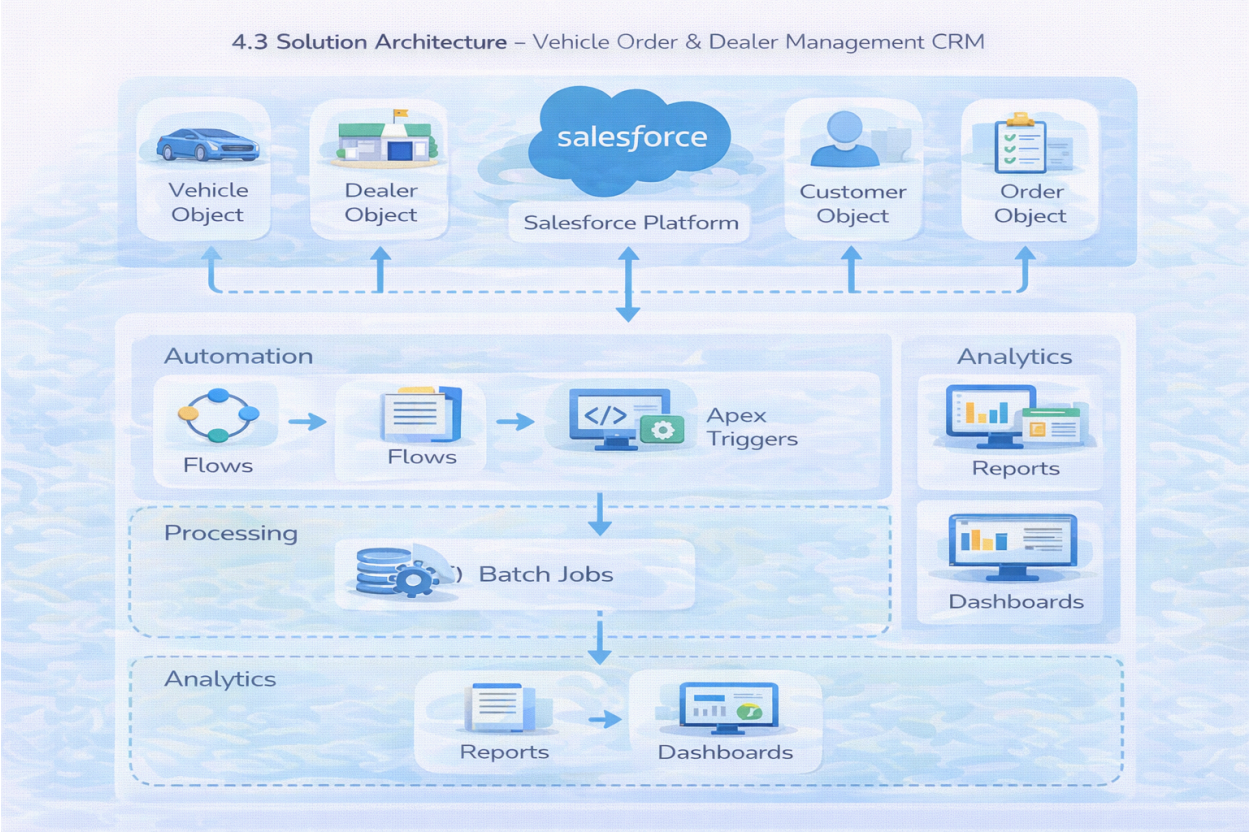
Parameter	Description

Problem	Manual vehicle ordering
Solution	Salesforce CRM
Innovation	Automated dealer assignment
Impact	Faster service
Business Model	Dealer network
Scalability	Multi-location support

4.3 Solution Architecture

Components:

- Vehicle Object
- Dealer Object
- Customer Object
- Order Object
- Test Drive Object
- Service Request Object
- Flows
- Apex Triggers
- Batch Jobs
- Reports
- Dashboards



5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

- Agile methodology
- Epics → Stories → Story Points
- Sprint-based execution
- Velocity calculation used for estimation

Product Backlog, Sprint Schedule, and Estimation (4 Marks)

Use the below template to create product backlog and sprint schedule

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members
Sprint-1	Developer Setup	USN-1	As a system administrator, I want to create and configure a Salesforce developer account so that I can build and deploy the vehicle order management system.	3	High	Member1
Sprint-2	Data Modeling	USN-2	As an admin, I want to create Vehicle, Dealer, Customer, Order,	5	High	Member2

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members
			and Service Request objects with proper relationships so that all business data is stored and managed systematically.			
Sprint-2	Data Modeling	USN-3	As a staff member, I want dedicated tabs and a Lightning App for all modules so that I can easily navigate and manage vehicle orders and dealer operations.	5	High	Member3
Sprint-3	Automation	USN-4	As a system, I want to validate stock availability, mandatory fields, and order inputs so that invalid or incomplete vehicle orders are prevented.	3	High	Member4
Sprint-3	Automation	USN-5	As a sales coordinator, I want automated flows to assign the nearest dealer and update order status so that processing becomes faster and requires minimal manual effort.	3	High	Member4
Sprint-4	Apex	USN-6	As a system, I want Apex triggers to automatically perform actions such as order confirmation and dealer assignment so that critical processes occur instantly and accurately.	5	High	Member1
Sprint-4	Apex	USN-7	As an inventory manager, I want batch jobs to update vehicle stock levels automatically so that inventory data remains accurate and up to date.	5	High	Member2
Sprint-5	Reports	USN-8	As a manager, I want reports showing order status, dealer performance, and sales metrics so that I can monitor operations and make informed decisions.	4	High	Member3
Sprint-6	Dashboards	USN-9	As a manager, I want dashboards showing order status, dealer performance, and sales metrics so that I can monitor operations and make informed decisions.	4	Medium	Member4

Project Tracker, Velocity & Burndown Chart: (4 Marks)

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed (as on Planned End Date)	Sprint Release Date (Actual)
Sprint-1	20	6 Days	24 Oct 2022	29 Oct 2022	20	29 Oct 2022
Sprint-2	20	6 Days	31 Oct 2022	05 Nov 2022		
Sprint-3	20	6 Days	07 Nov 2022	12 Nov 2022		
Sprint-4	20	6 Days	14 Nov 2022	19 Nov 2022		

6. Project Development Phase:

What you'll learn

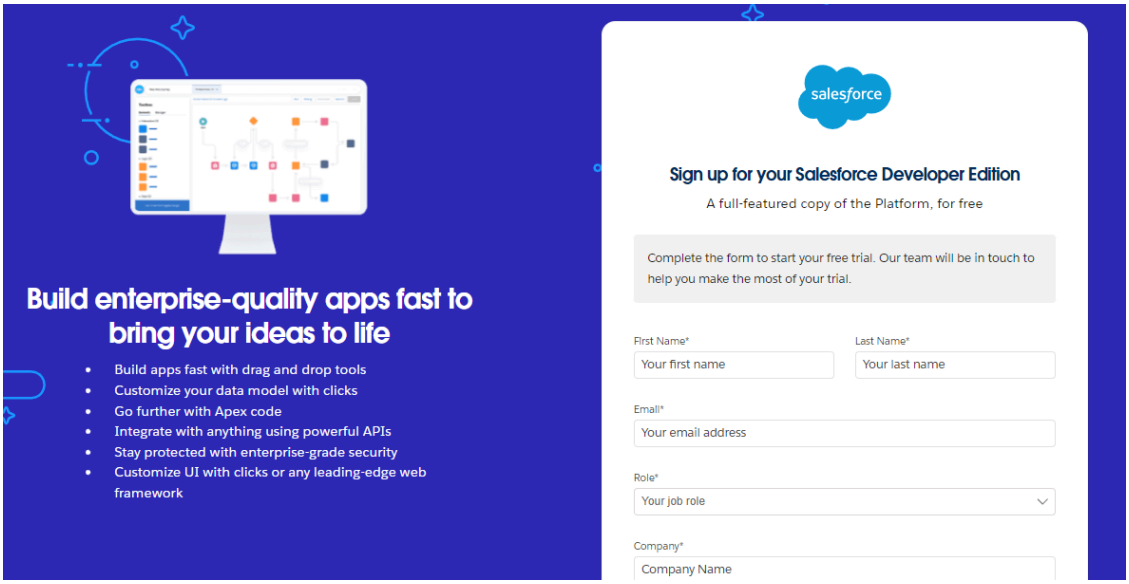
1. Data Modelling
2. Fields and Relationships
3. Lightning App Builder
4. Record Triggered Flows
5. Apex and Apex Triggers
6. Batch Apex
7. Scheduled Apex

Milestone 1: Salesforce Account

Activity 1: Creating Developer Account

Creating a developer org in salesforce.

1. Go to https://developer.salesforce.com/signup
2. On the sign up form, enter the following details :



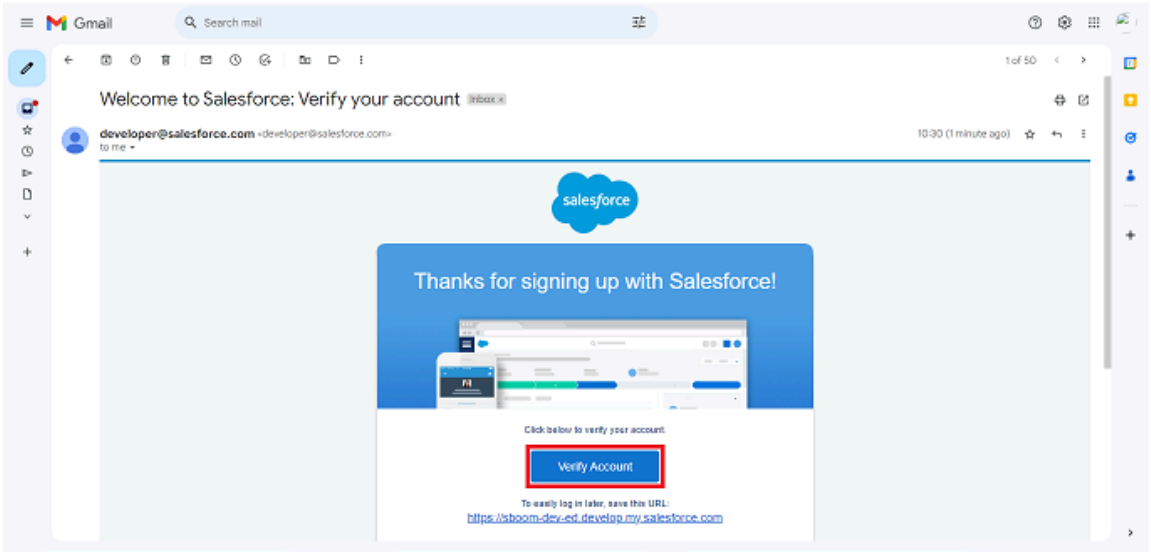
1) First name & Last name
2) Email
3) Role : Developer
4) Company : College Name
5) County : India
6) Postal Code : pin code
7) Username : should be a combination of your name and company

This need not be an actual email id, you can give anything in the format : username@organization.com

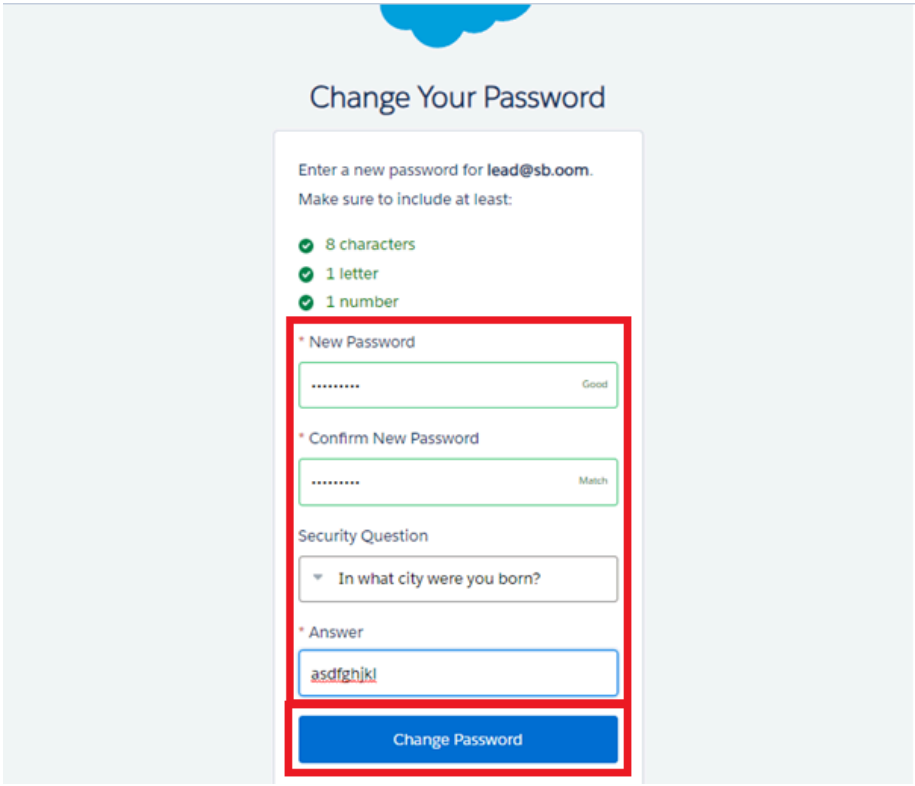
Click on sign me up after filling these.

Activity 2: Account Activation

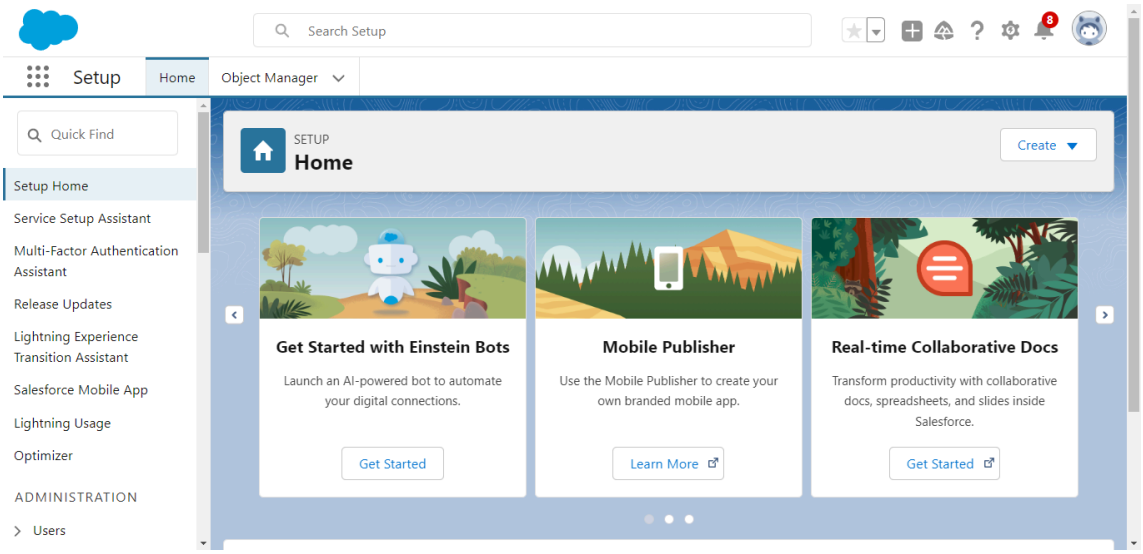
1. Go to the inbox of the email that you used while signing up. Click on the verify account to activate your account. The email may take 5-10mins.



2. Click on Verify Account
3. Give a password and answer a security question and click on change password.



4. Then you will redirect to your salesforce setup page.



Milestone 2: Data Management-Objects

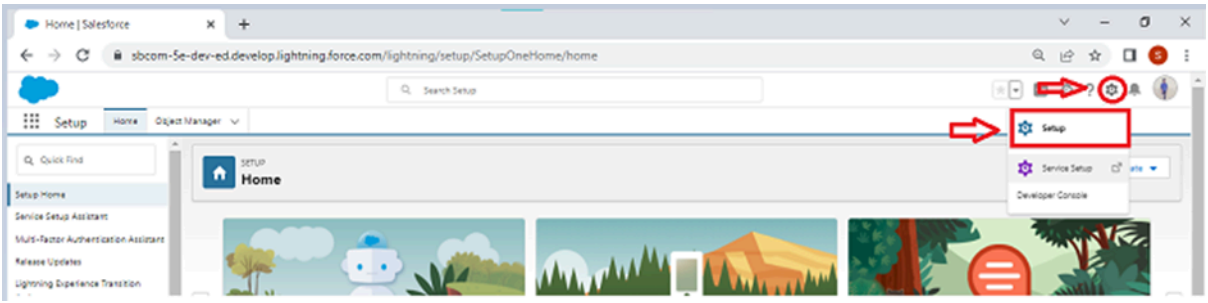
Objects & Relationships

Object Name	Purpose	Relationships
Vehicle__c	Stores vehicle details	Related to Dealer & Orders
Vehicle_Dealer__c	Stores authorized dealer info	Related to Orders
Vehicle_Customer__c	Stores customer details	Related to Orders & Test Drives
Vehicle_Order__c	Tracks vehicle purchases	Related to Customer & Vehicle

Vehicle_Test_Drive__c	Tracks test drive bookings	Related to Customer & Vehicle
Vehicle_Service_Request__c	Tracks vehicle servicing requests	Related to Customer & Vehicle

To Navigate to Setup page:

Click on gear icon → click setup.



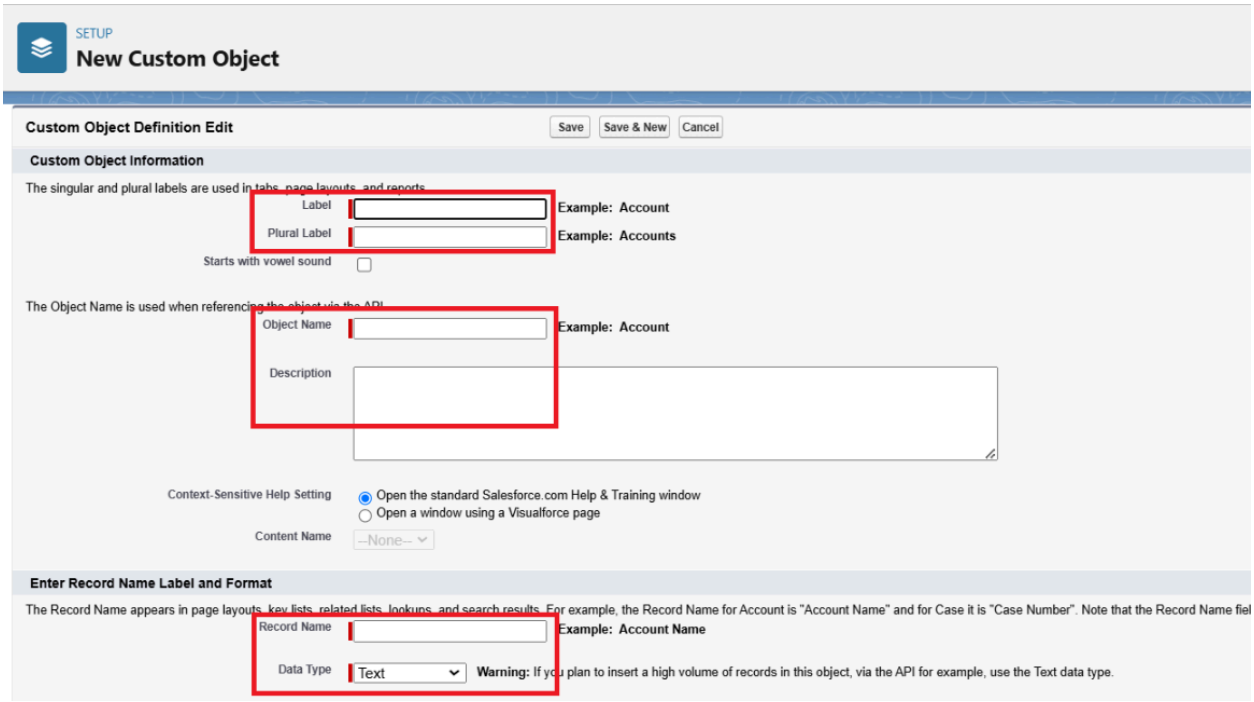
Activity 1: Create Vehicle Object

The purpose of creating a Vehicle custom object is to have a clear picture of the vehicle details.

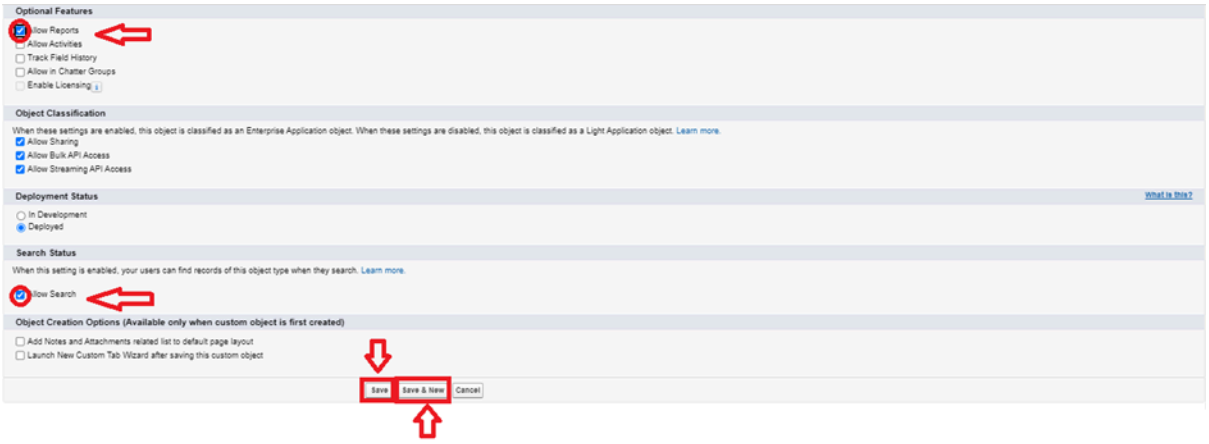
To create an object:

1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.

1) Enter the label name→ Vehicle
2) Plural label name→ Vehicles
3) Enter Record Name Label and Format
    - Record Name → Vehicle Name
    - Data Type → Text



2. Click on Allow reports,
3. Allow search → **Save.**



Activity 2: Create Dealer Object

The purpose of creating a Dealer object is to have detailed information about Dealer.

To create an object:

1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.
    1) Enter the label name→ Vehicle Dealer
    2) Plural label name→ Vehicle Dealers
    3) Enter Record Name Label and Format
        - Record Name → Dealer Name
        - Data Type → Text
2. Click on Allow reports,
3. Allow search → **Save**

Activity 3: Creating remaining objects

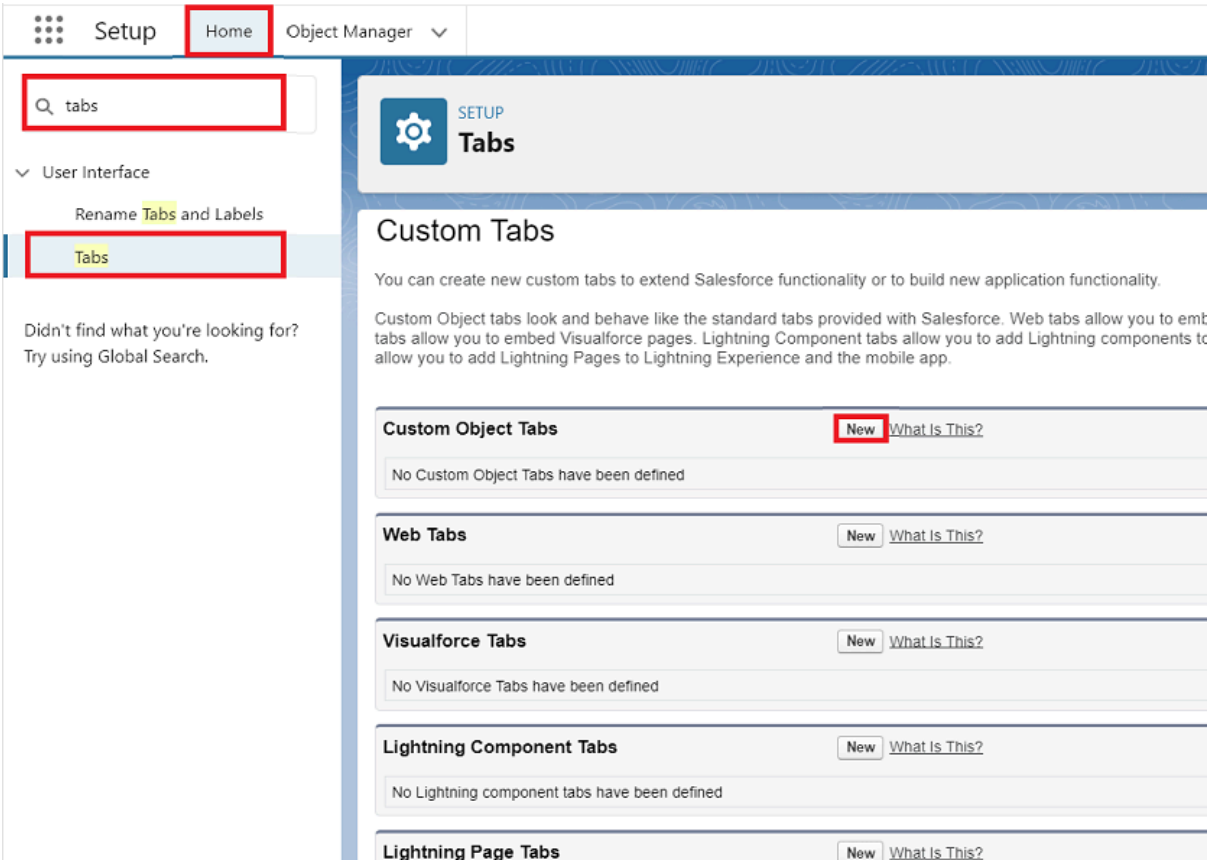
Creating remaining objects

Create remaining objects which have mentioned in the above table Data Management-Objects

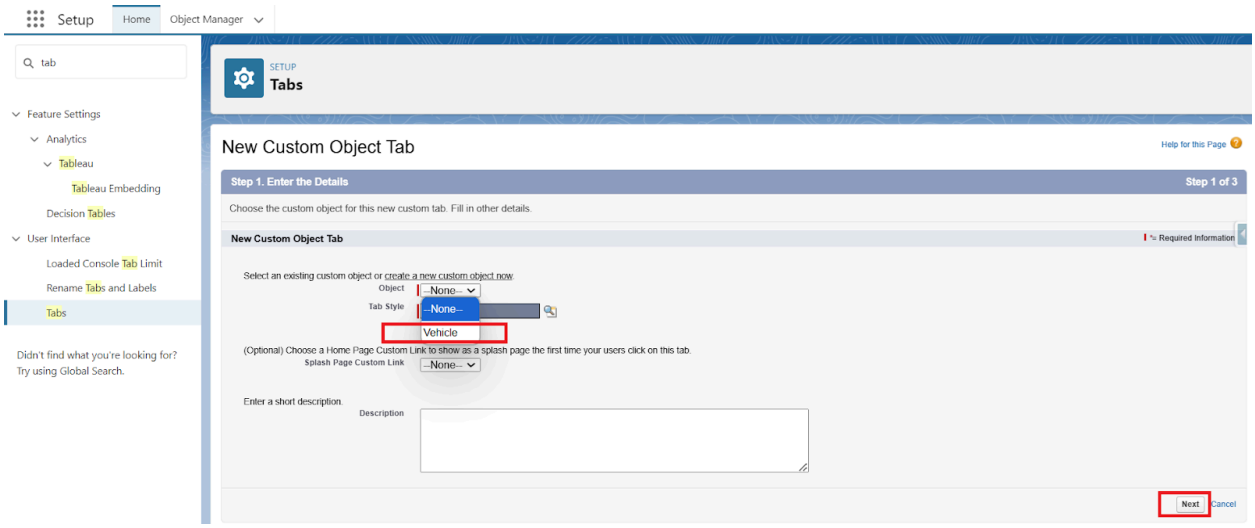
Milestone 3: Data Management-Tabs

Activity 1: Creating a Custom Tab(Vehicle)

1. Go to setup page → type Tabs in Quick Find bar → click on tabs → New (under custom object tab)



2. Select Object(Vehicle) → Select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default → Save.



Activity 2:

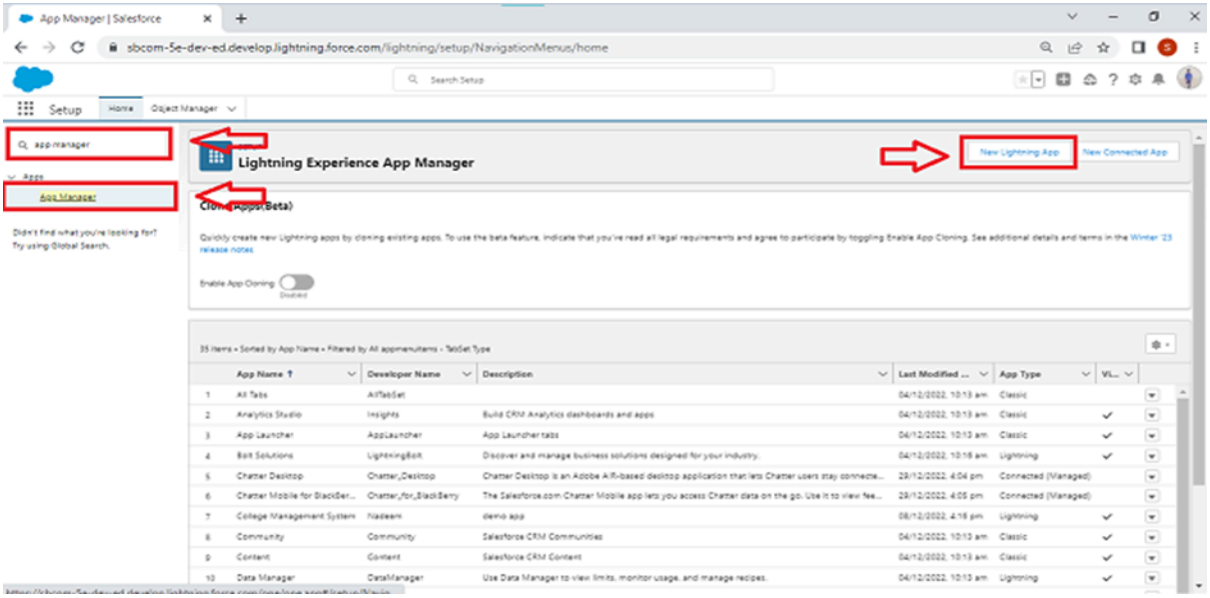
Now create tabs for Other objects as well which we have created above Objects.

Milestone 4: Data Management-App Manager

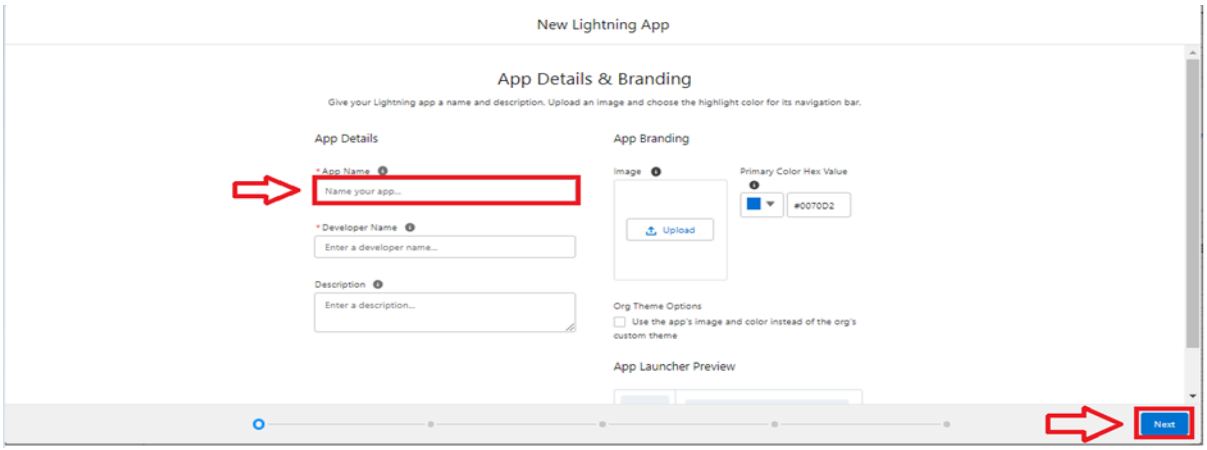
Activity 1: Create a Lightning App

To create a lightning app page:

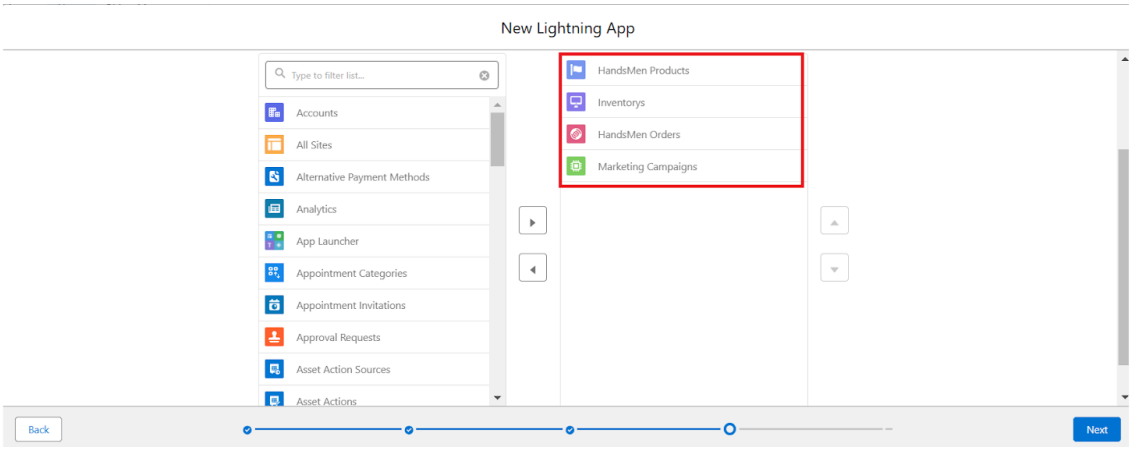
1. Go to setup page → search “app manager” in quick find → select “app manager” → click on New lightning App.



2. Fill the app name in app details and branding as follow
   App Name : WhatNext Vision Motors
   Developer Name : this will auto populated
   Description : Give a meaningful description
   Image : optional (if you want to give any image you can otherwise not mandatory)
   Primary color hex value : keep this default
3. Then click Next → (App option page) keep it as default → Next → (Utility Items) keep it as default → Next.

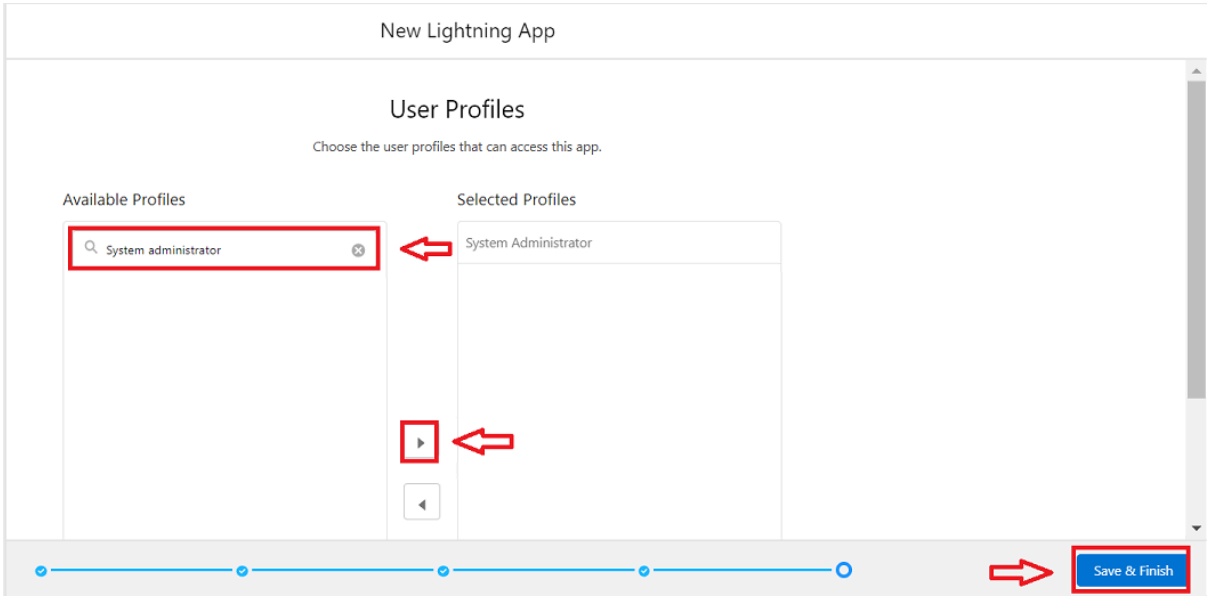


4. To Add Navigation Items:



   Search the items in the search bar(Vehicle, Dealer, Customer, Order, Test Drive, Service Request, Reports, Dashboard) from the search bar and move it using the arrow button → Next.
   Note: select the custom object which we have created in the previous activity.

5. To Add User Profiles:



Search profiles (System administrator) in the search bar → click on the arrow button → save & finish.

Milestone 5: Data Management-Fields

Key Fields for Each Object

1. Vehicle__c (Custom Object)

- **Vehicle_Name__c** (Text)
- **Vehicle_Model__c** (Picklist: Sedan, SUV, EV, etc.)
- **Stock_Quantity__c** (Number)
- **Price__c** (Currency)
- **Dealer__c** (Lookup to Dealer__c)
- **Status__c** (Picklist: Available, Out of Stock, Discontinued)

2.Vehicle_ Dealer__c (Custom Object)

- **Dealer_Name__c** (Text)
- **Dealer_Location__c** (Text)
- **Dealer_Code__c** (Auto Number)
- **Phone__c** (Phone)
- **Email__c** (Email)

3. Vehicle_Order__c (Custom Object)

- **Customer__c** (Lookup to Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Order_Date__c** (Date)
- **Status__c** (Picklist: Pending, Confirmed, Delivered, Canceled)

4. Vehicle_Customer__c (Custom Object)

- **Customer_Name__c** (Text)
- **Email__c** (Email)
- **Phone__c** (Phone)
- **Address__c** (Text)
- **Preferred_Vehicle_Type__c** (Picklist: Sedan, SUV, EV, etc.)

5. Vehicle_Test_Drive__c (Custom Object)

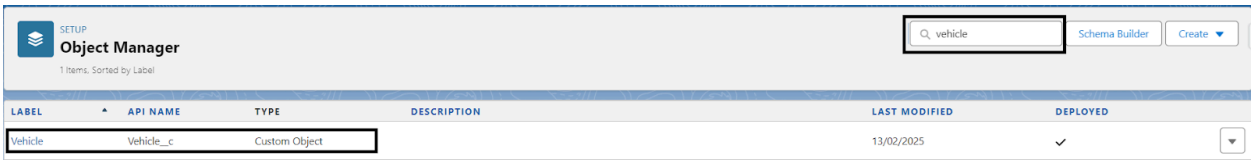
- **Customer__c** (Lookup to Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Test_Drive_Date__c** (Date)
- **Status__c** (Picklist: Scheduled, Completed, Canceled)

6. Vehicle_Service_Request__c (Custom Object)

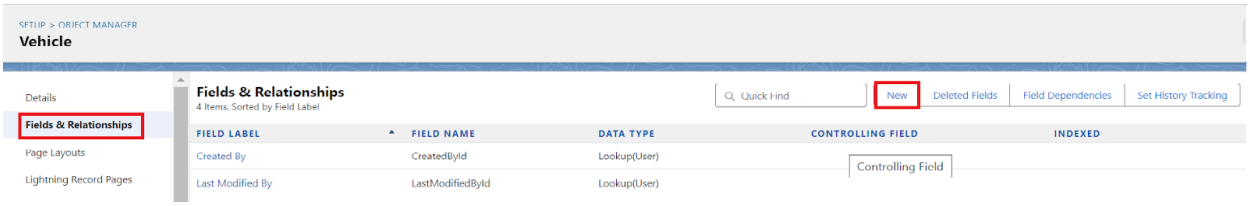
- **Customer__c** (Lookup to Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Service_Date__c** (Date)
- **Issue_Description__c** (Text)
- **Status__c** (Picklist: Requested, In Progress, Completed)

Object Field Creation :

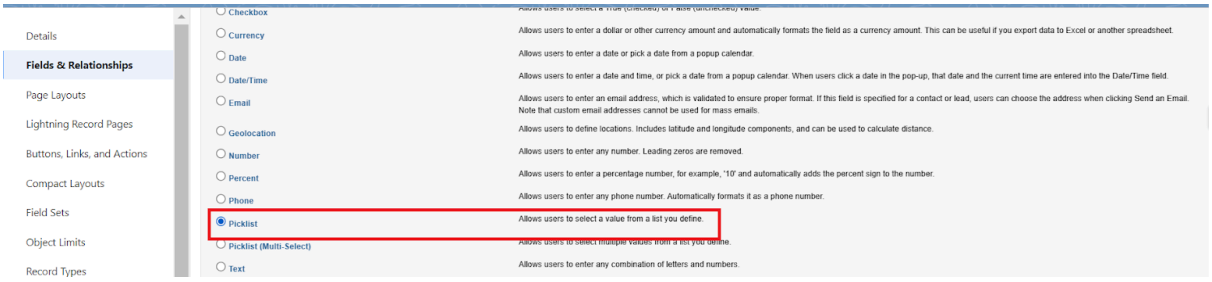
1. Go to setup → click on Object Manager → type object name(Vehicle) in quick find bar→ click on the object.



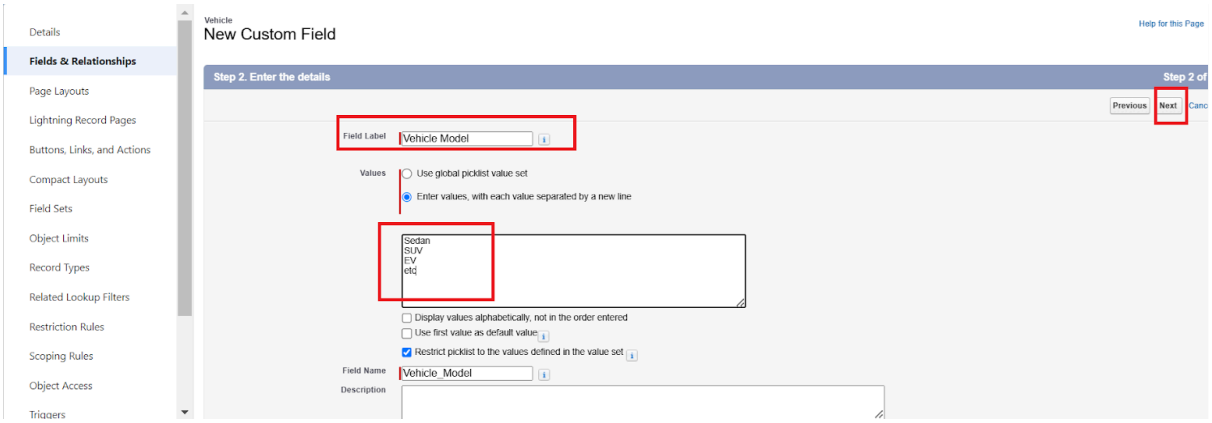
2. Now click on “Fields & Relationships” → New



3. Select Data type as “PickList”.



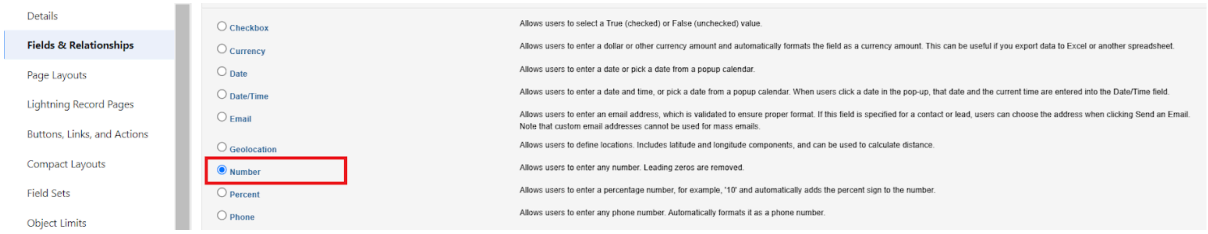
4. Click on Next



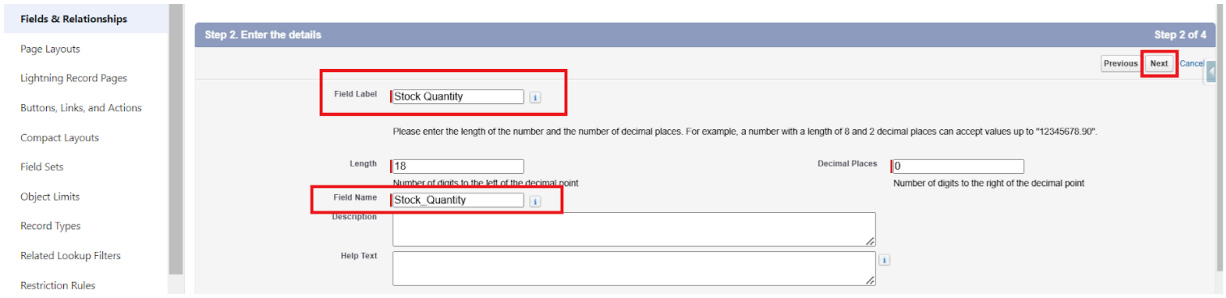
5. Fill the above as following:
   - Field Label: Vehicle Modal
   - Fill PickList Values
   - Click on Next → Next → Save and new.

Object Field Creation-2 :

1. Repeat the steps 1 & 2 from the activity 1
2. Select Data type as “Number”.



3. Click on Next

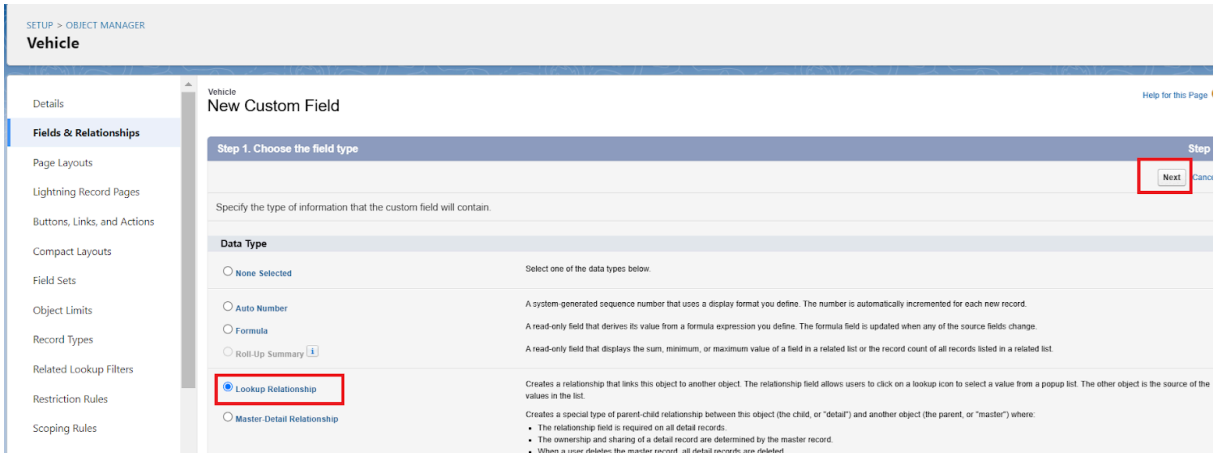


4. Fill the above as following:
   - Field Label: Stock Quality
   - Click on Next → Next → Save and new.

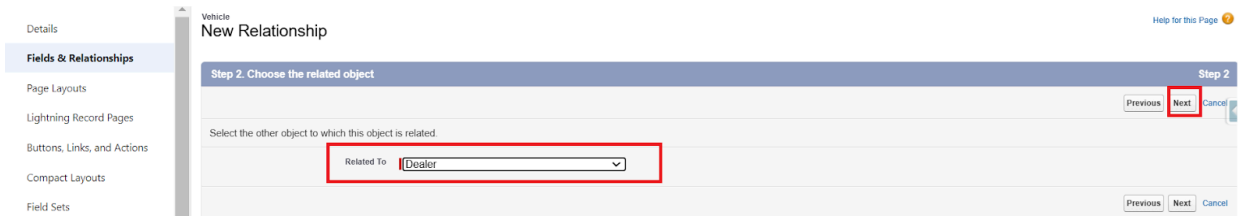
Relationship Field Creation :

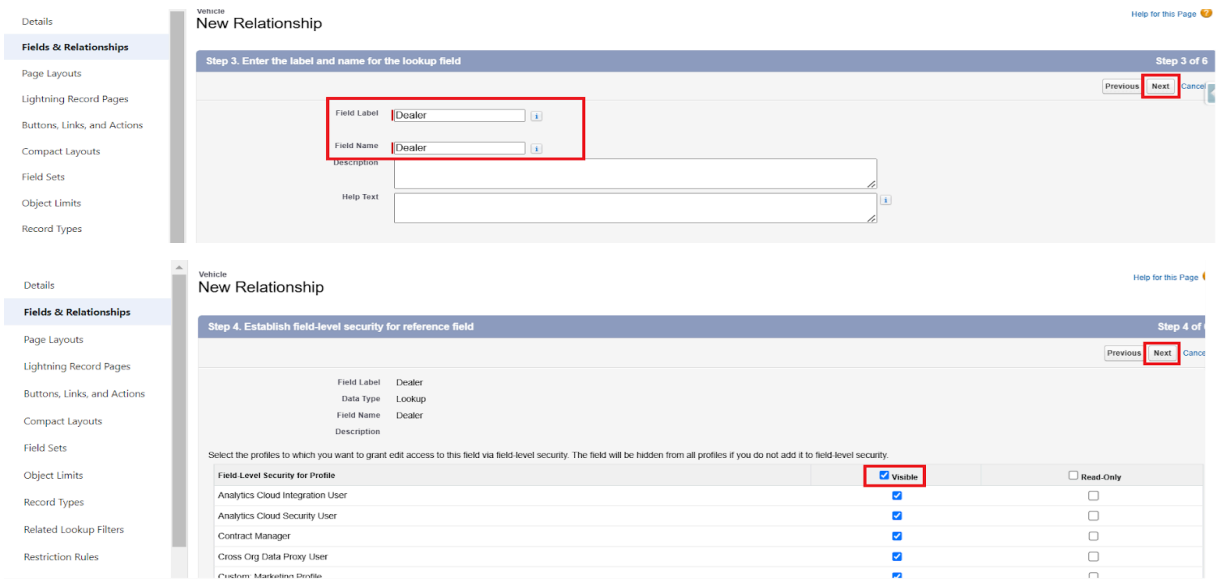
Creating a Lookup RelationShip in Vehicle Object to Dealer Object.

1. Select DataType as Lookup Relationship.



2. Click Next
3. Select Dealer Object Form Related Filed.





4. Click Next and Save.

More Fields Creation

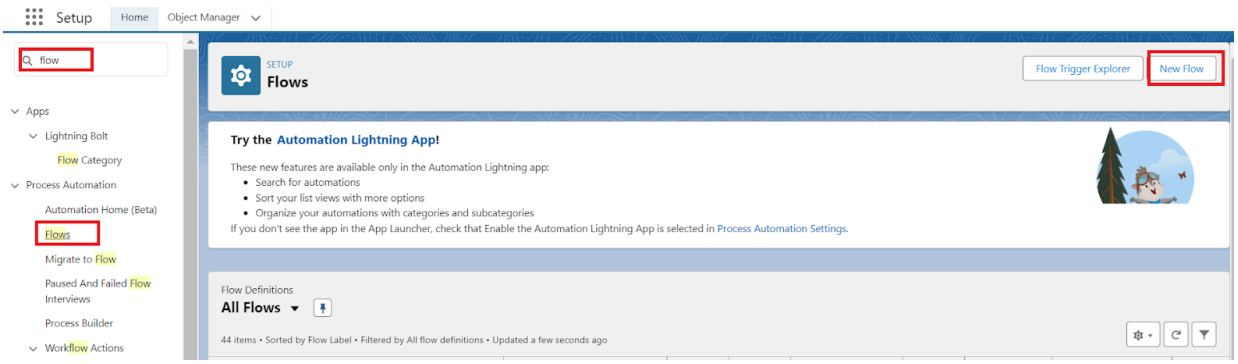
Create remaining fields as mentioned in the Data Management-Fields

Milestone 6: Automation

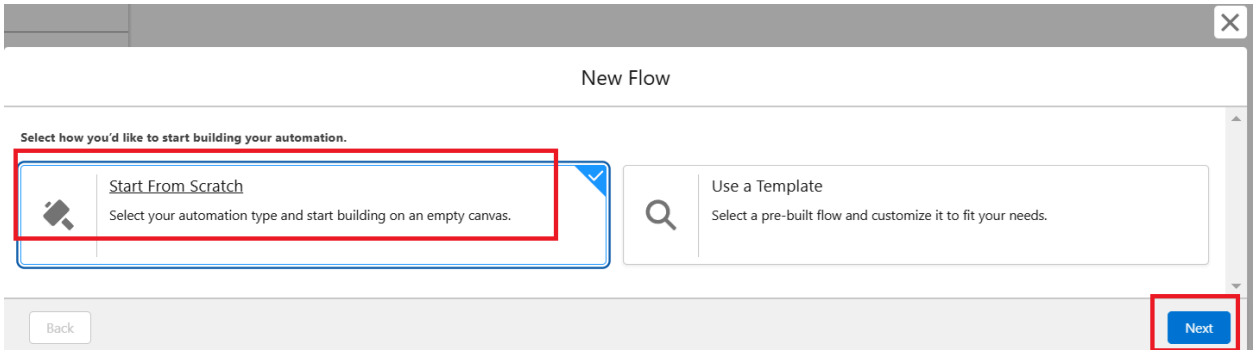
Flow Creation :

Creating a record triggered flow to assign nearest dealer to the customer's location

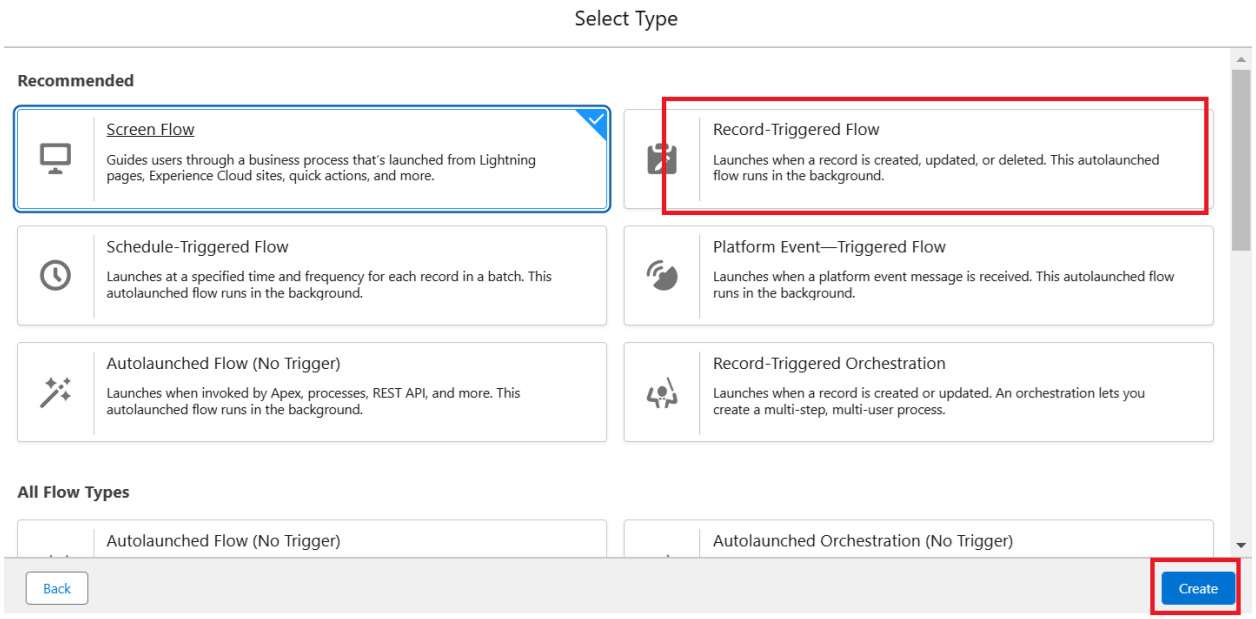
Step 1 : In Quick Find, type Flows and click on Flows. Click New Flow.



Step 2 : Select Start From Scratch and click Next.



Step 3 : Select Record-Triggered Flow and click Create



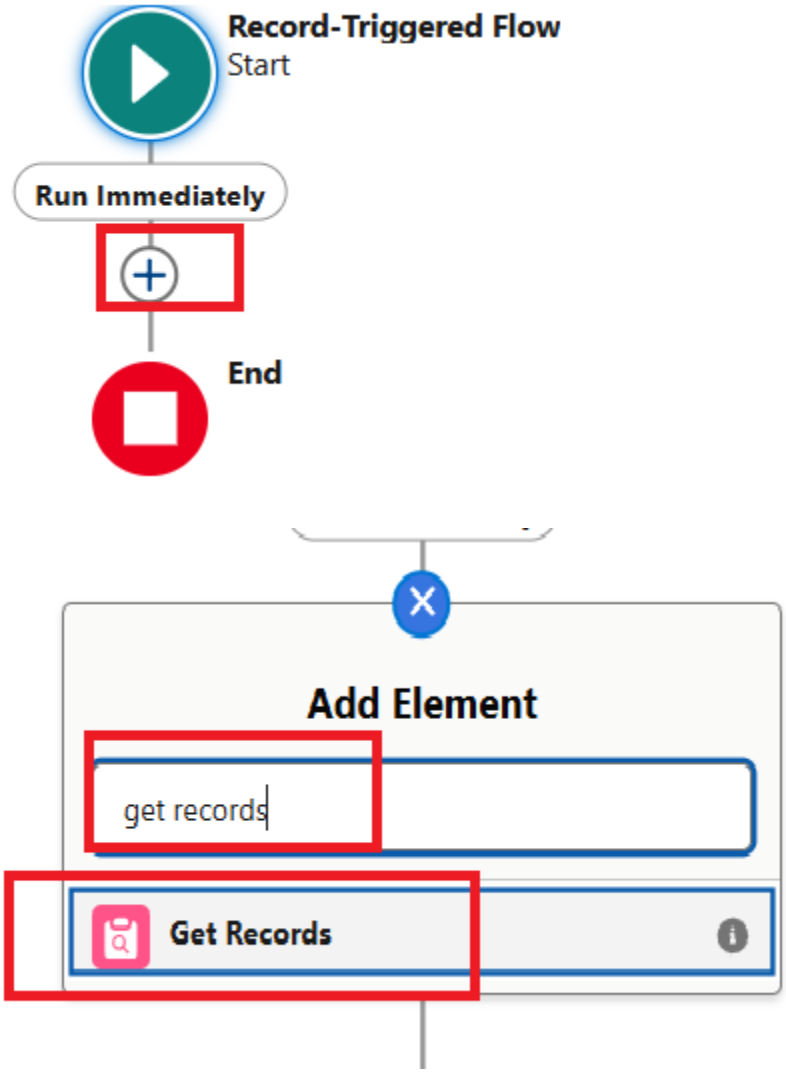
Step 4 : Select Vehicle Order Object

Trigger the Flow When : Select Record is Created.

Set Entry Condition :
All Conditions Are Met (AND)
Filed : Status__c
Operator : Equals
Value : Pending

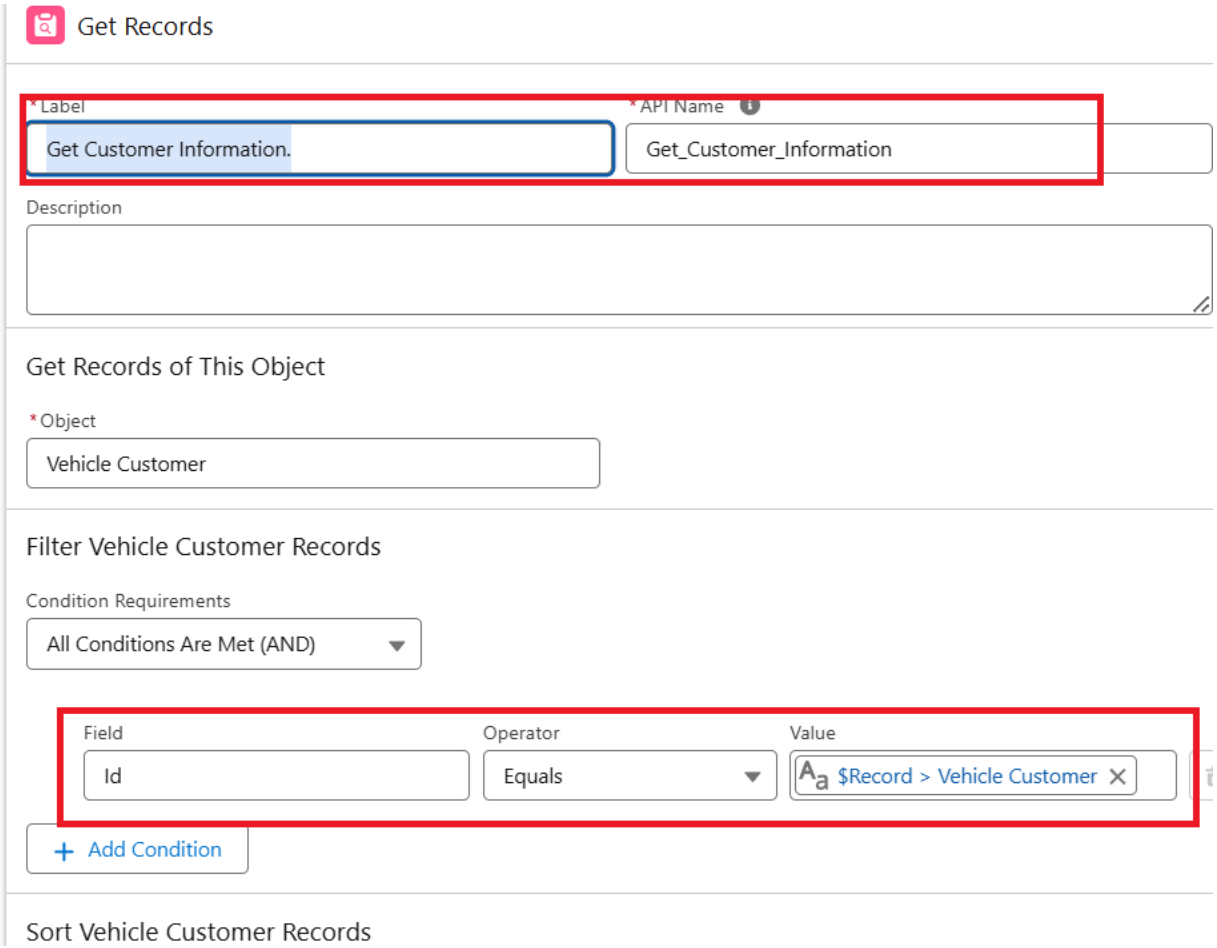
```
app = Flask(__name__)
```

Step 5 : Click + → Select **Get Records**



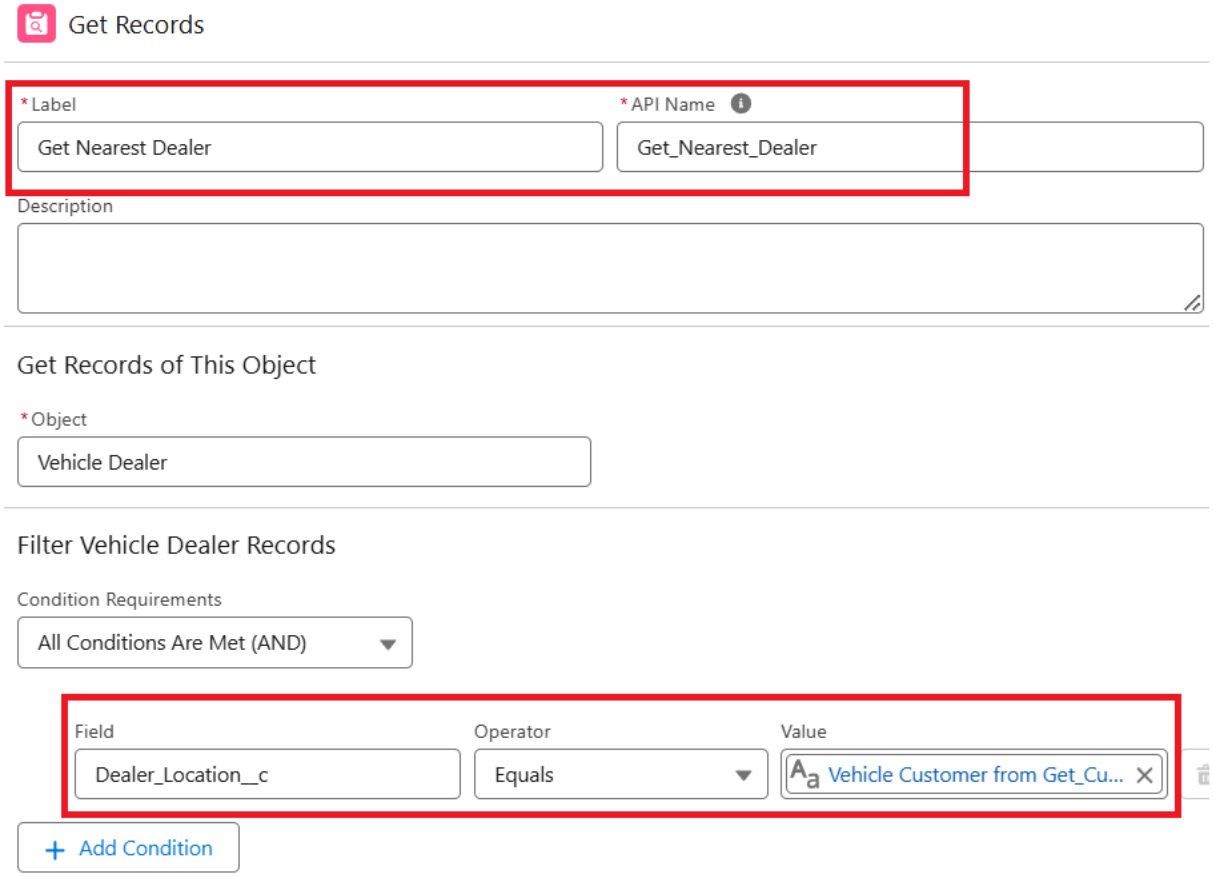
Step 6 : **Label :** Get Customer Information

- Object : Vehicle Customer
- Condition :
- Field : Id
- Operator : Equals
- Value : {!$Record.Vehicle_Customer__c}



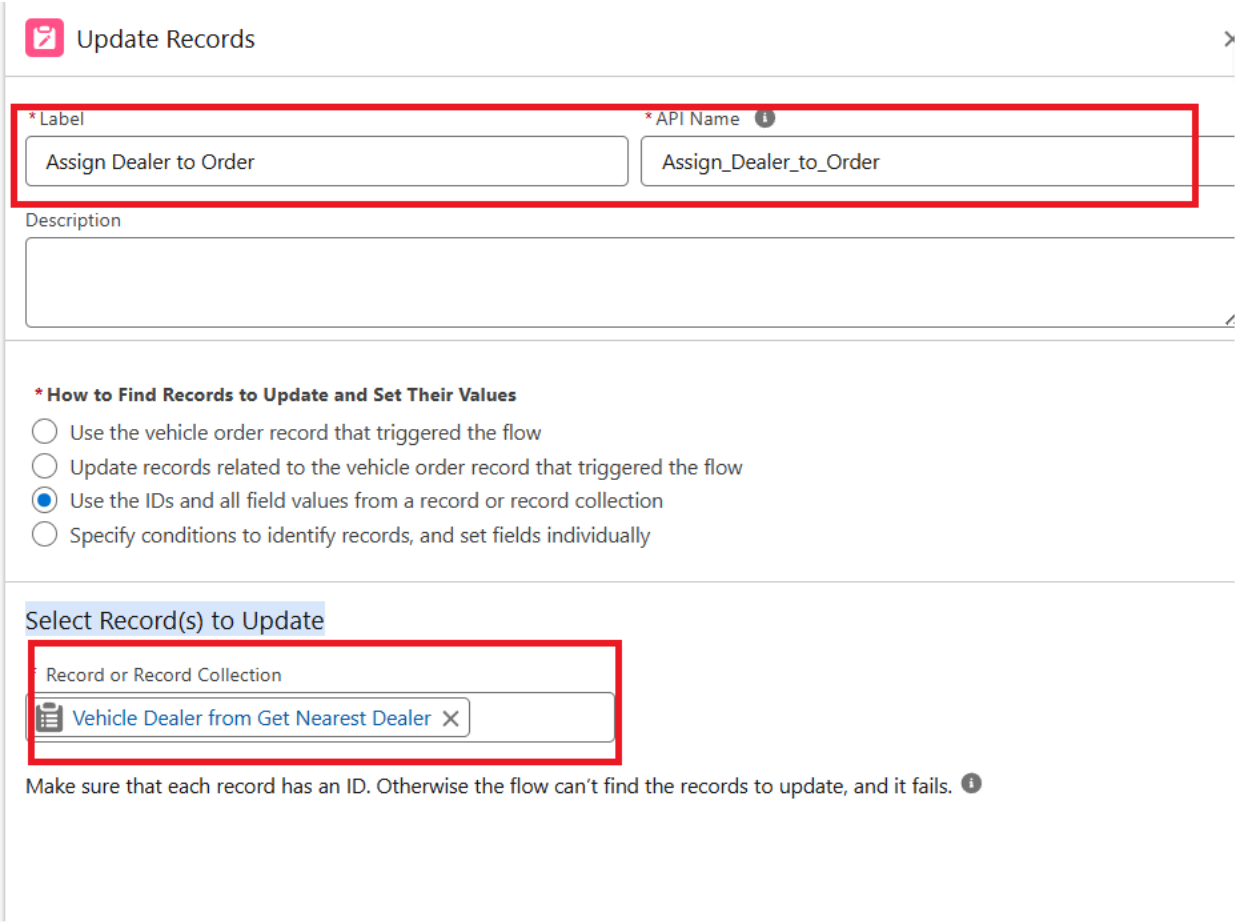
Step 7 : Click + → Select **Get Records**

- **Label :** Get Nearest Dealer
- Object : Vehicle Dealer
- Condition :
- Field : Dealer_Location__c
- Operator : Equals
- Value : {!Get_Customer_Information.Address__c}



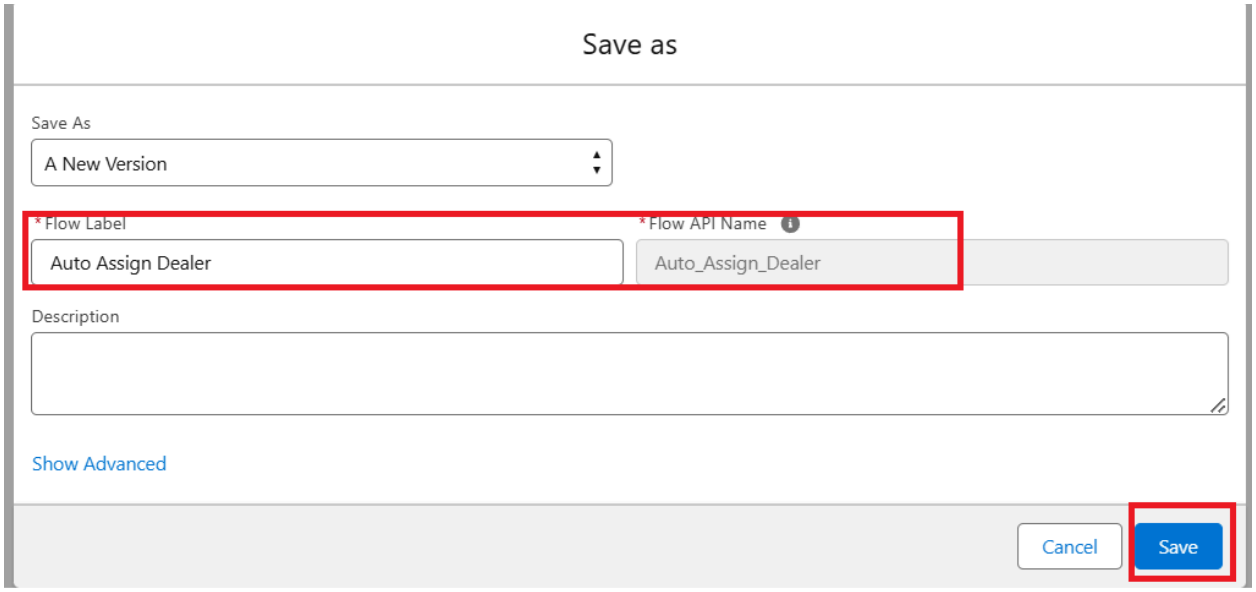
Step 8 : Click + → Select **Update Records**

- **Label :** Assign Dealer to Order
- *How to Find Records to Update and Set Their Values : Use the IDs and all field values from a record or record collection
- Select Record(s) to Update : {!Get_Nearest_Dealer}

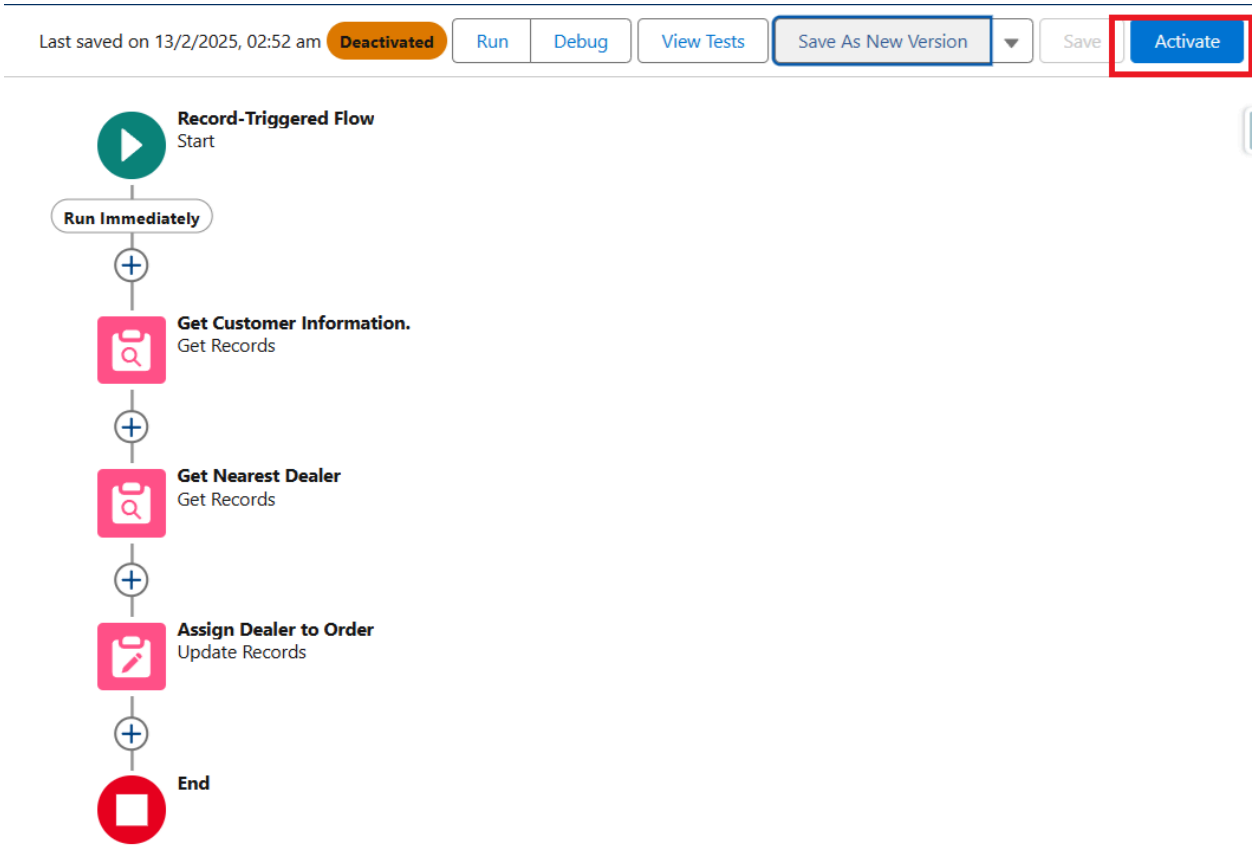


Step 9 : Click Save and Give label Name and Activate Flow.

- Label Name : Auto Assign Dealer



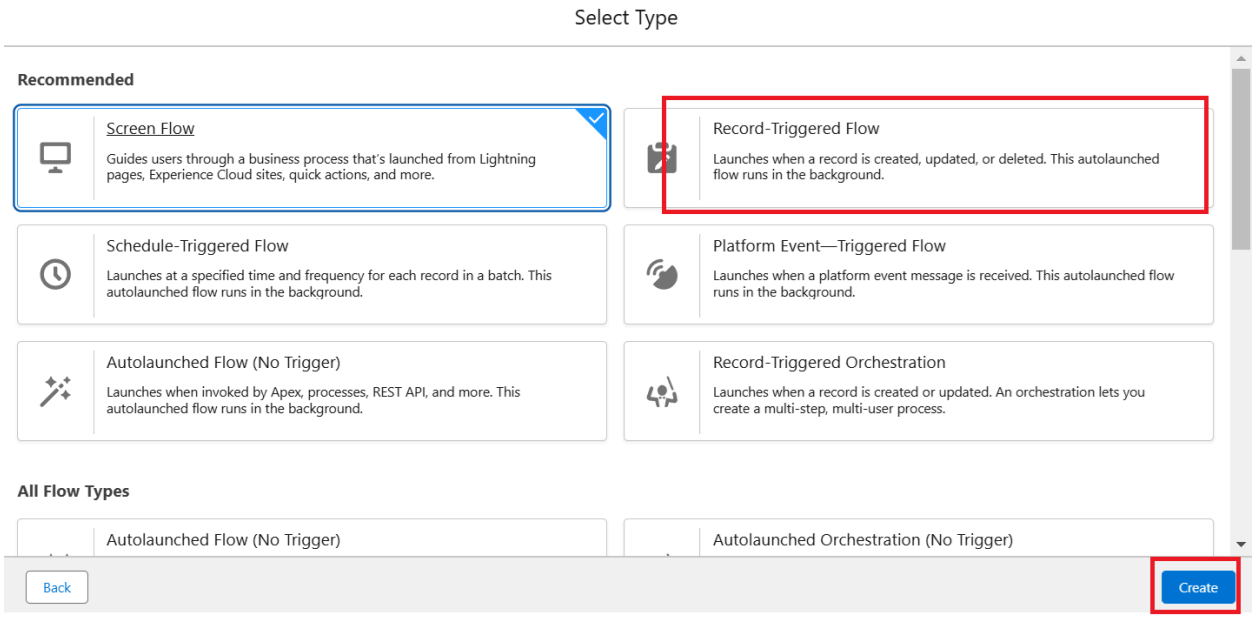
Step 10 : Activate Flow



Record-Trigger Flow Creation :

Creating a record triggered flow to send an email to the customer reminding about the test drive.

Step 1 : Select Record-Triggered Flow and click Create



Step 2 : Select Vehicle Test Drive Object

Trigger the Flow When : A record is created or updated

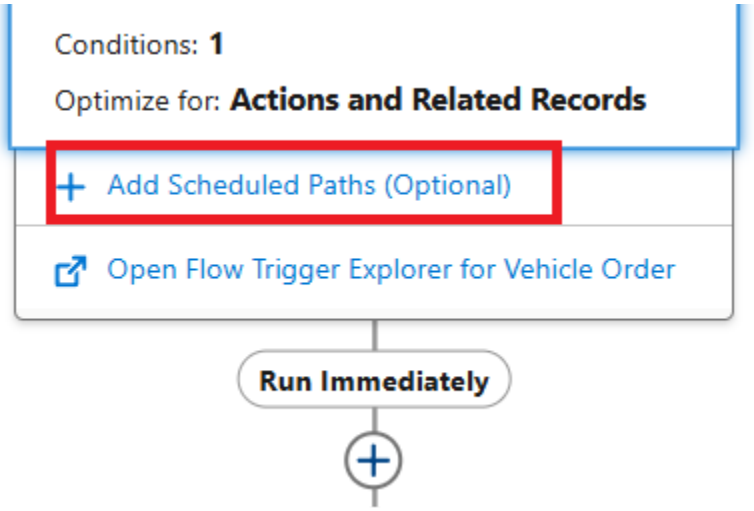
Set Entry Condition :

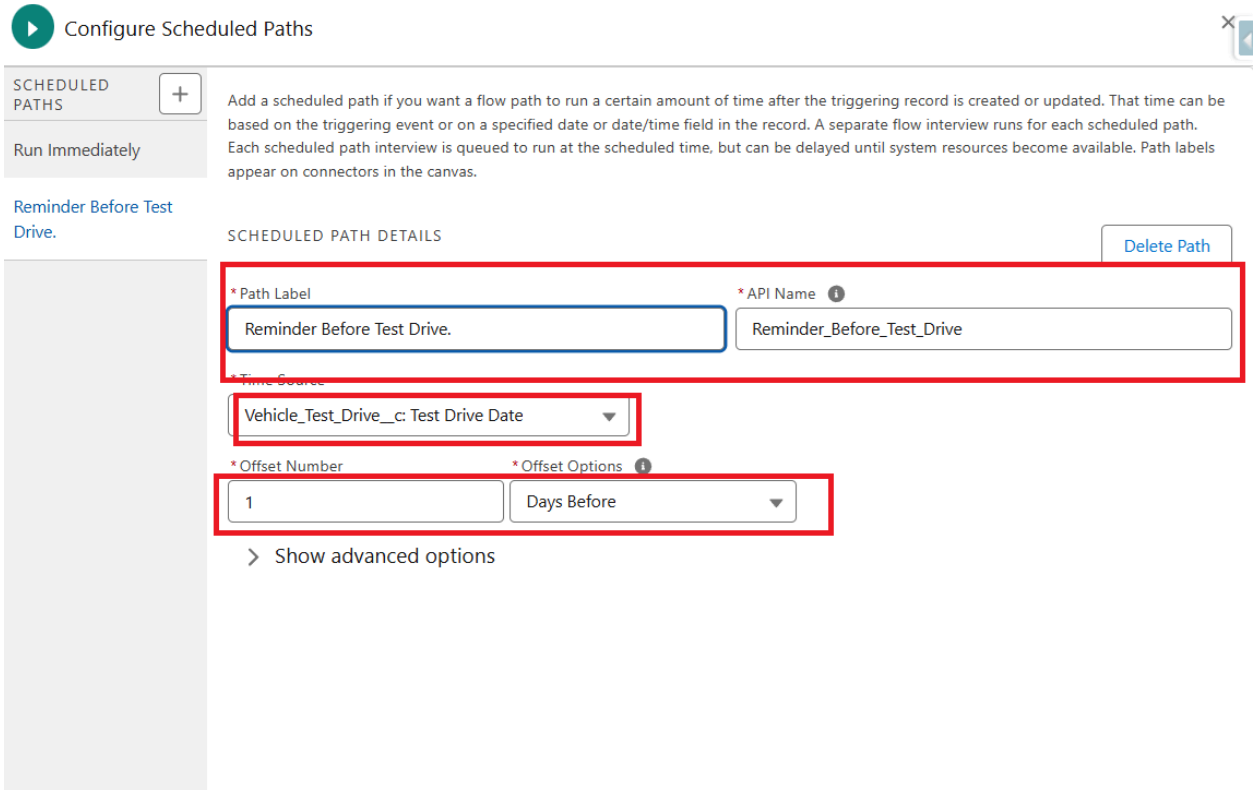
- All Conditions Are Met (AND)
- Filed : Status__c
- Operator : Equals
- Value : Scheduled

Step 3 : Click + Add Scheduled Paths (below the trigger).

- Label: Reminder Before Test Drive.
- Time Source: Test_Drive_Date__c
- Offset Number: 1.
- Offset Options: Days Before.

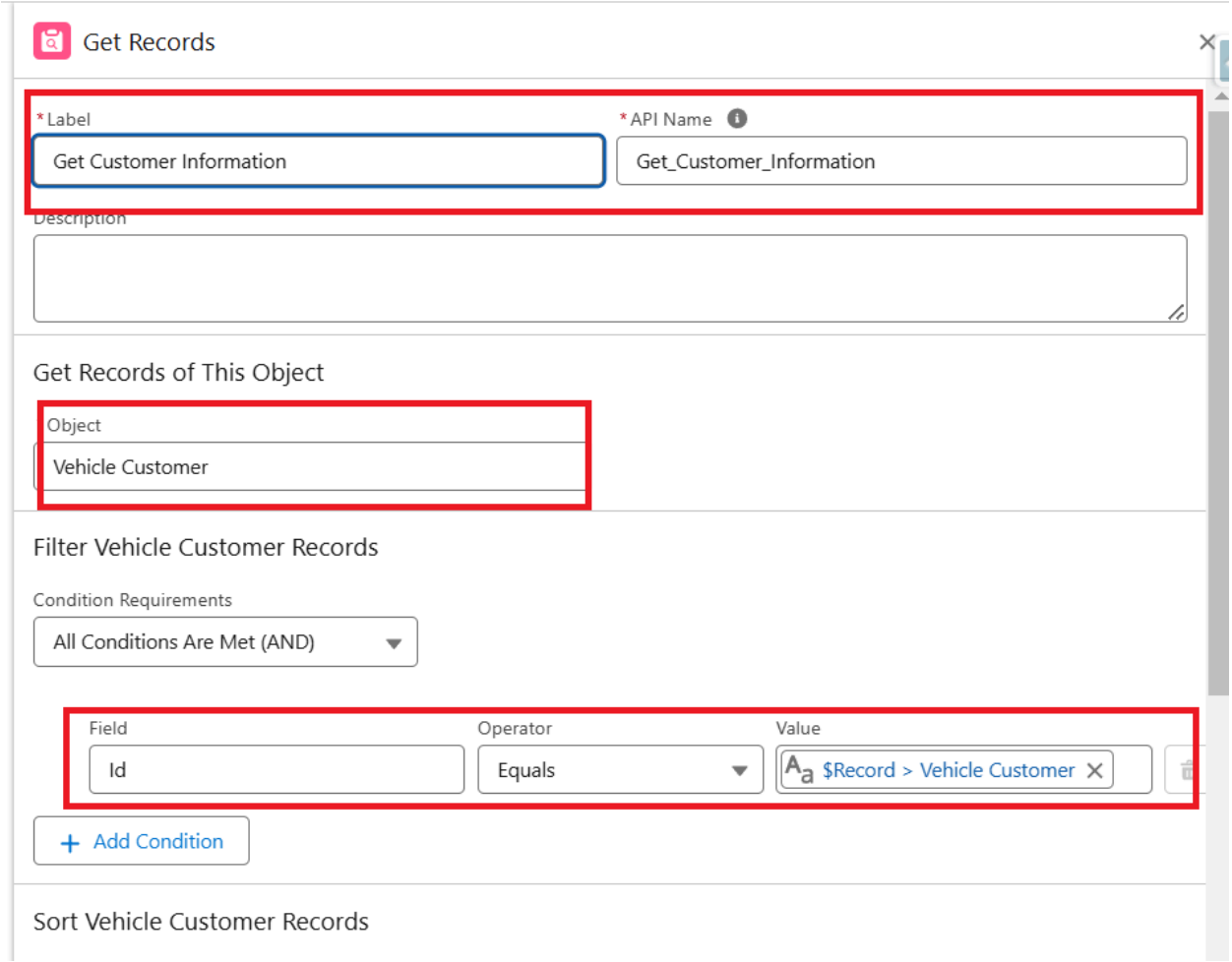
- Click Done.





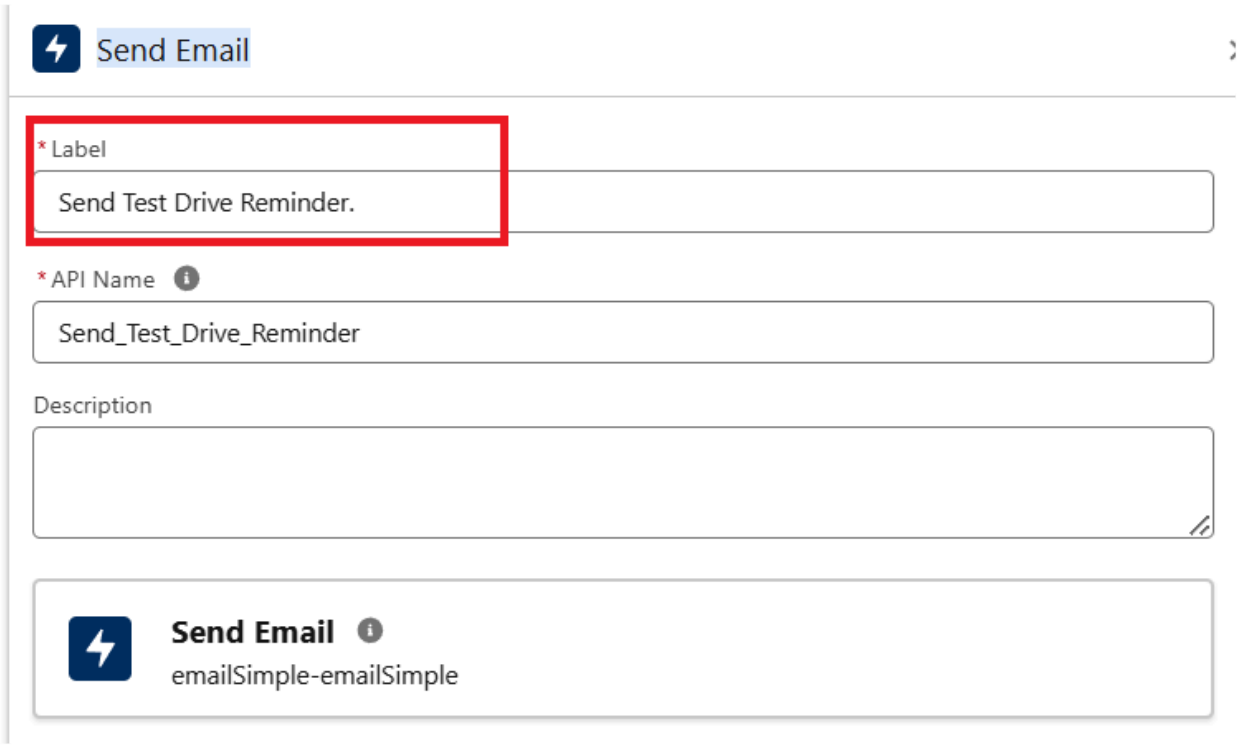
Step 4 :

1. Click + Add Element → Get Records.
2. Label: Get Customer Information.
3. Object: Vehicle_Customer__c.
4. Filter Conditions: Id = {!$Record.Customer__c}.
5. How Many Records to Store: Select Only the first record.
6. How to Store Record Data: Choose Automatically store all fields.



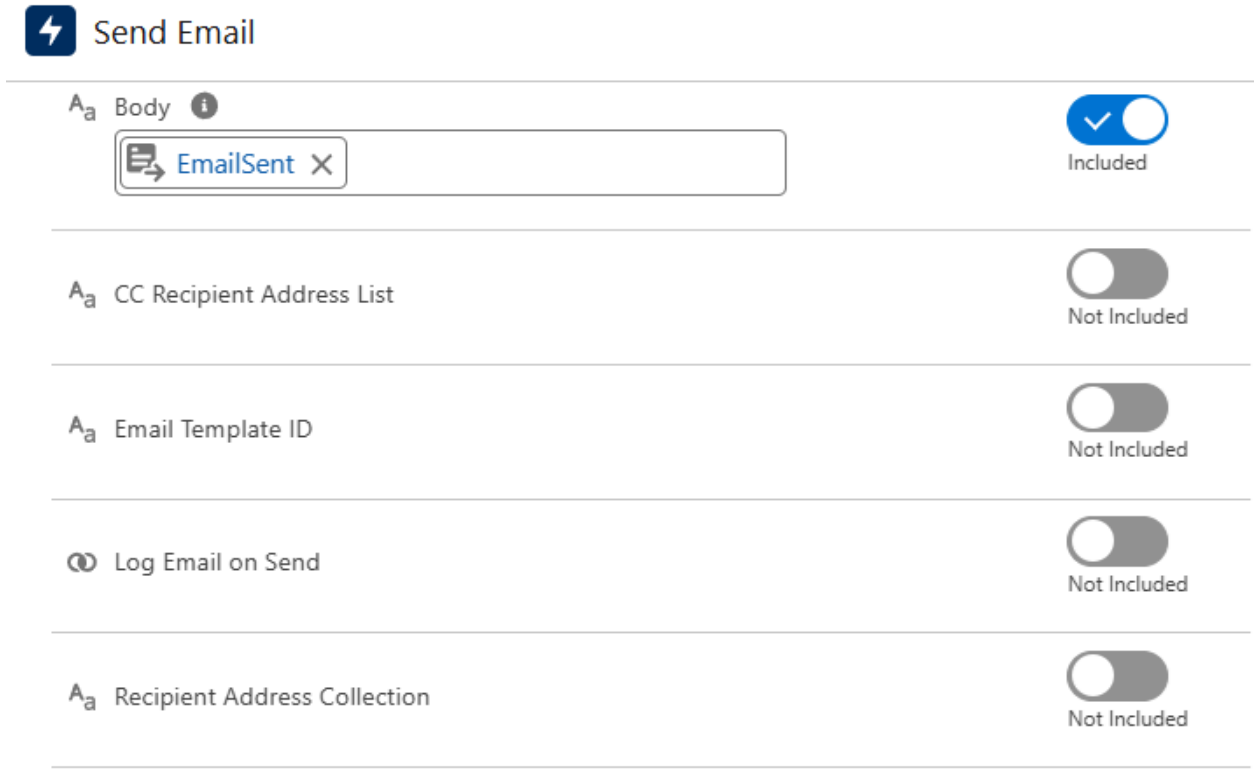
Step 5: Send Reminder Email

1. Click + Add Element → Action.
2. Action Type: Send Email
3. Label: Send Test Drive Reminder.
4. Subject: "Reminder: Your Test Drive is Tomorrow!".
5. Recipient Address List : {!Get_Customer_Information.Email__c}
6. Rich-Text-Formatted Body : True



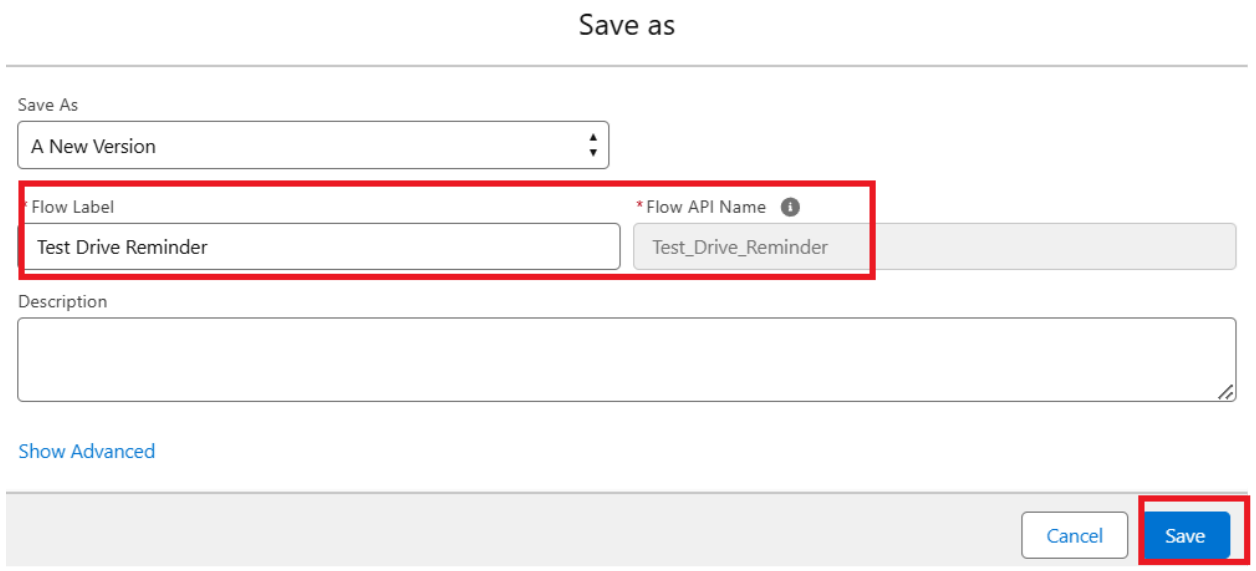
7. Body : Create Variable

8 . Api Name : EmailSent



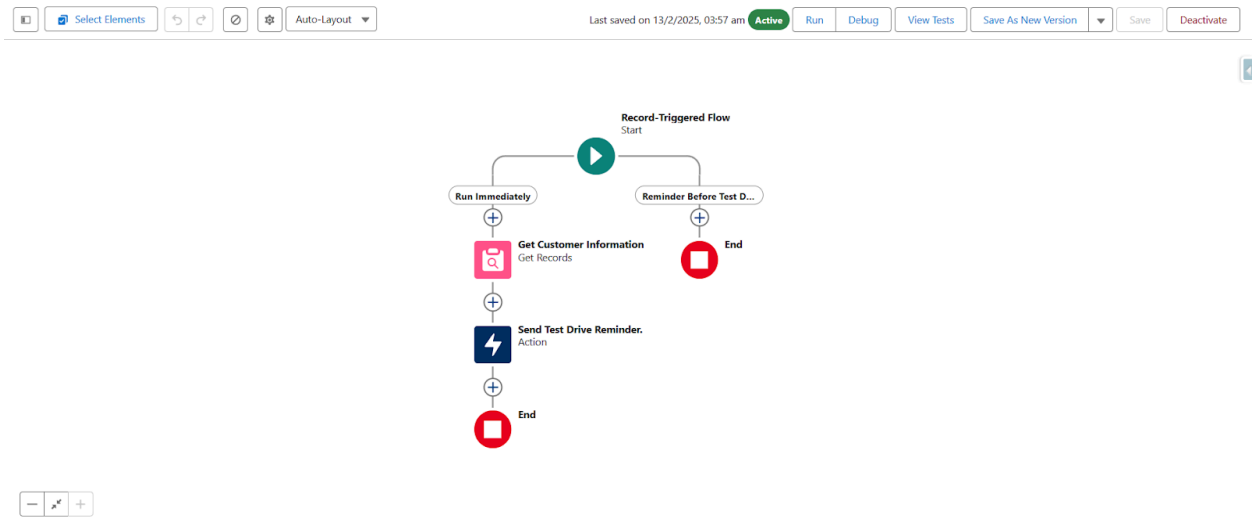
Step 6 : Click save

- **Label Name :** Test Drive Reminder

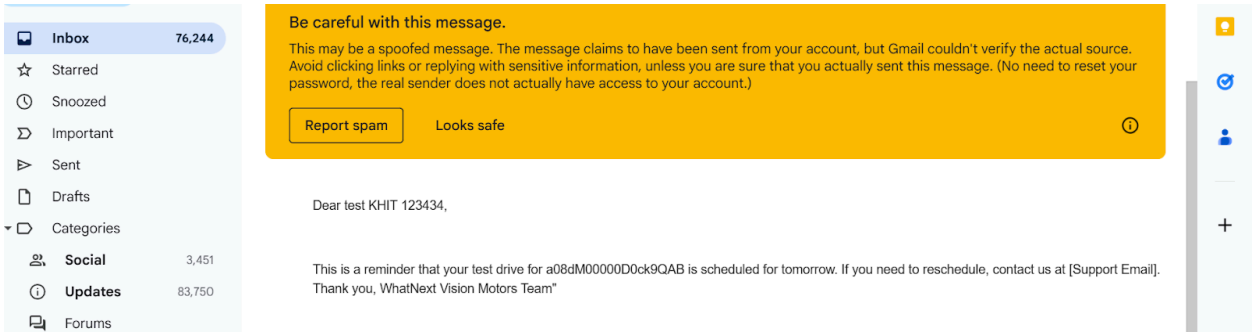


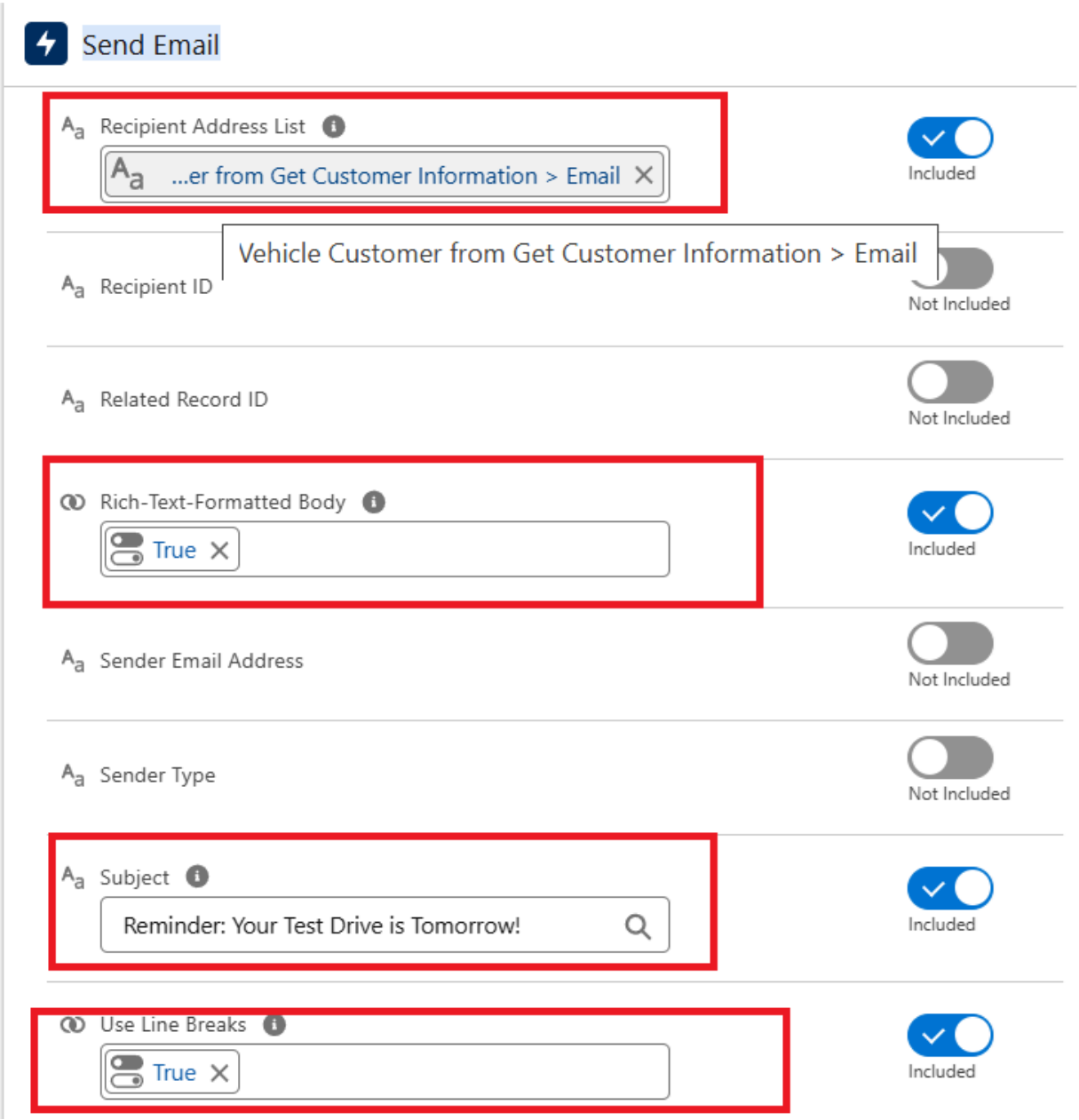
Step 7 : Activate Flow

Flow Page After Creation :



Test your Flow :

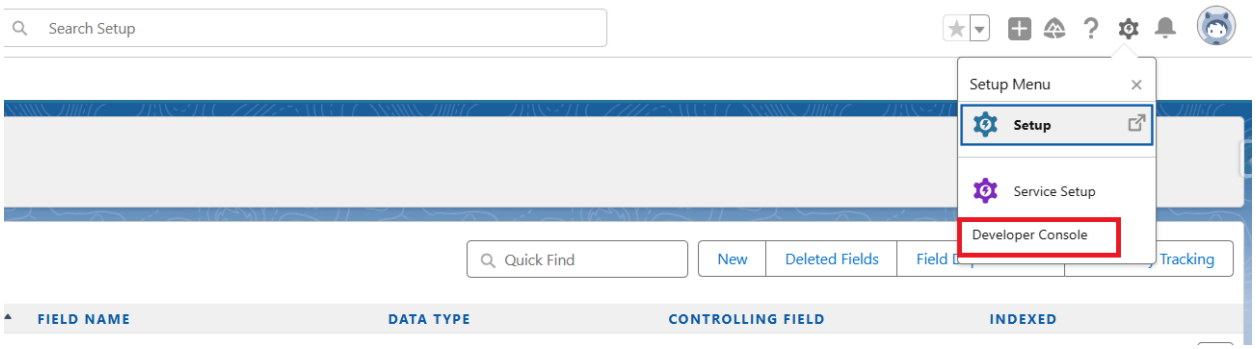

Inbox 76,244
Starred
Snoozed
Important
Sent
Drafts
Categories
Social 3,451
Updates 83,750
Forums
Be careful with this message.
This may be a spoofed message. The message claims to have been sent from your account, but Gmail couldn't verify the actual source. Avoid clicking links or replying with sensitive information, unless you are sure that you actually sent this message. (No need to reset your password, the real sender does not actually have access to your account.)
Report spam
Looks safe
Dear test KHIT 123434,
This is a reminder that your test drive for a08dM00000D0ck9QAB is scheduled for tomorrow. If you need to reschedule, contact us at [Support Email].
Thank you, WhatNext Vision Motors Team"


Send Email
Recipient Address List
...er from Get Customer Information > Email
Included
Recipient ID
Vehicle Customer from Get Customer Information > Email
Not Included
Related Record ID
Not Included
Rich-Text-Formatted Body
True
Included
Sender Email Address
Not Included
Sender Type
Not Included
Subject
Reminder: Your Test Drive is Tomorrow!
Included
Use Line Breaks
True
Included

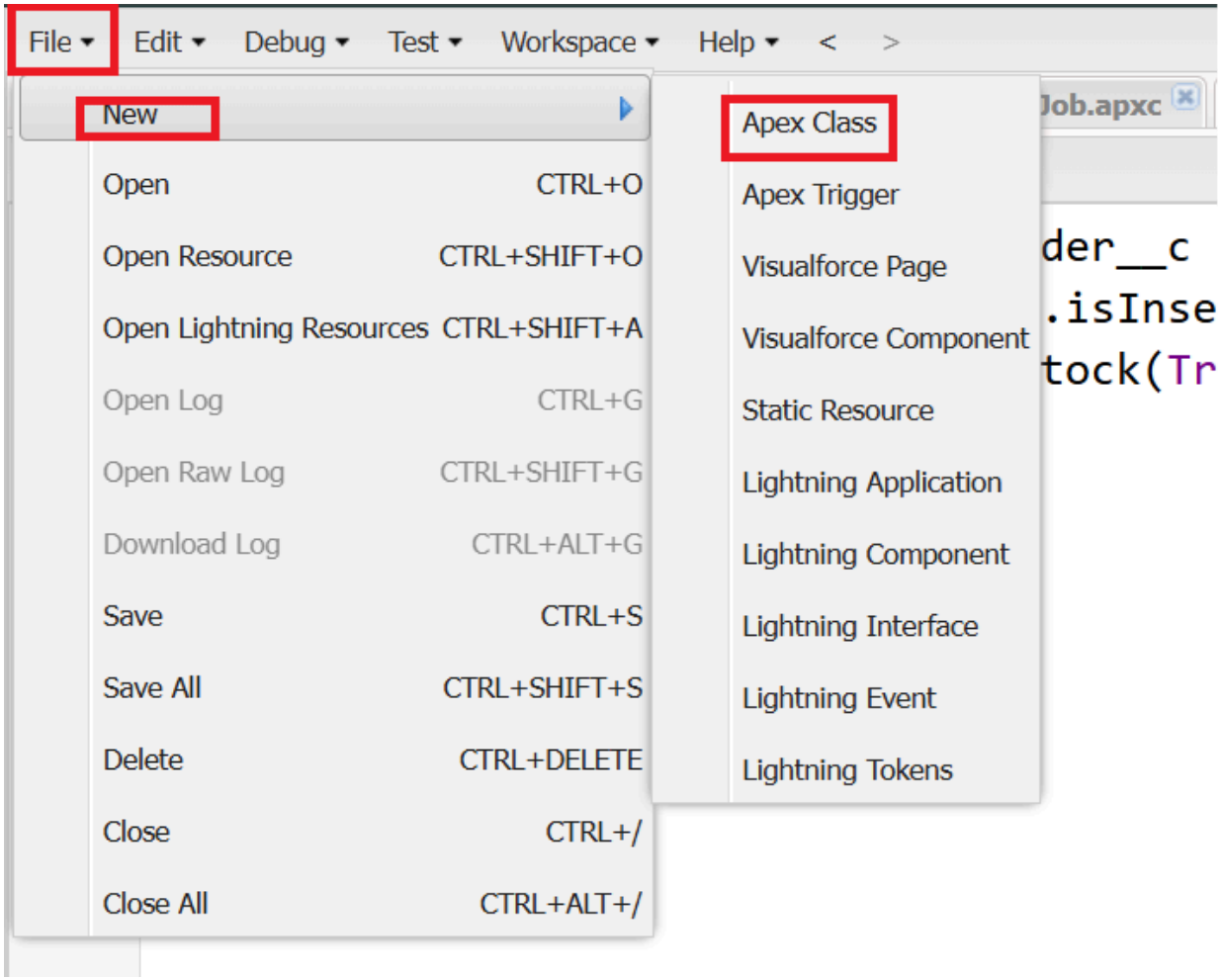
Milestone 7: Apex and Batch Class

Create Apex and Trigger Batch Jobs :

Step 1 : Click Developer Console From Gear icon

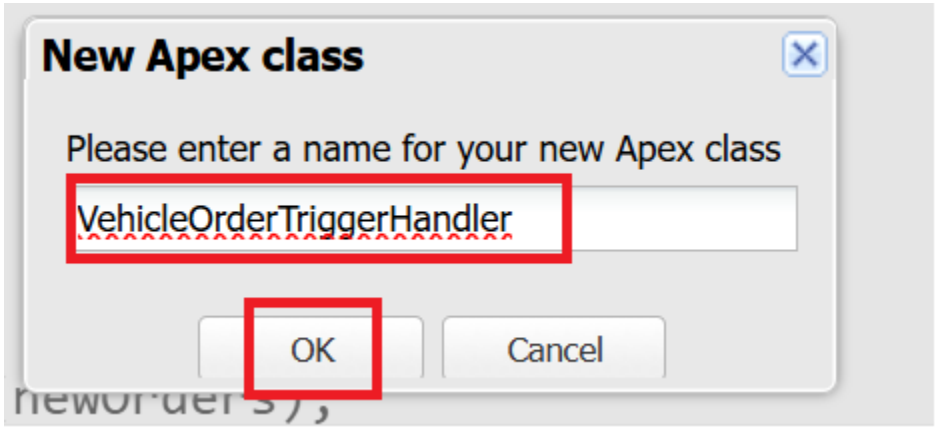


Step 2 : click File then Select New then Select Apex.

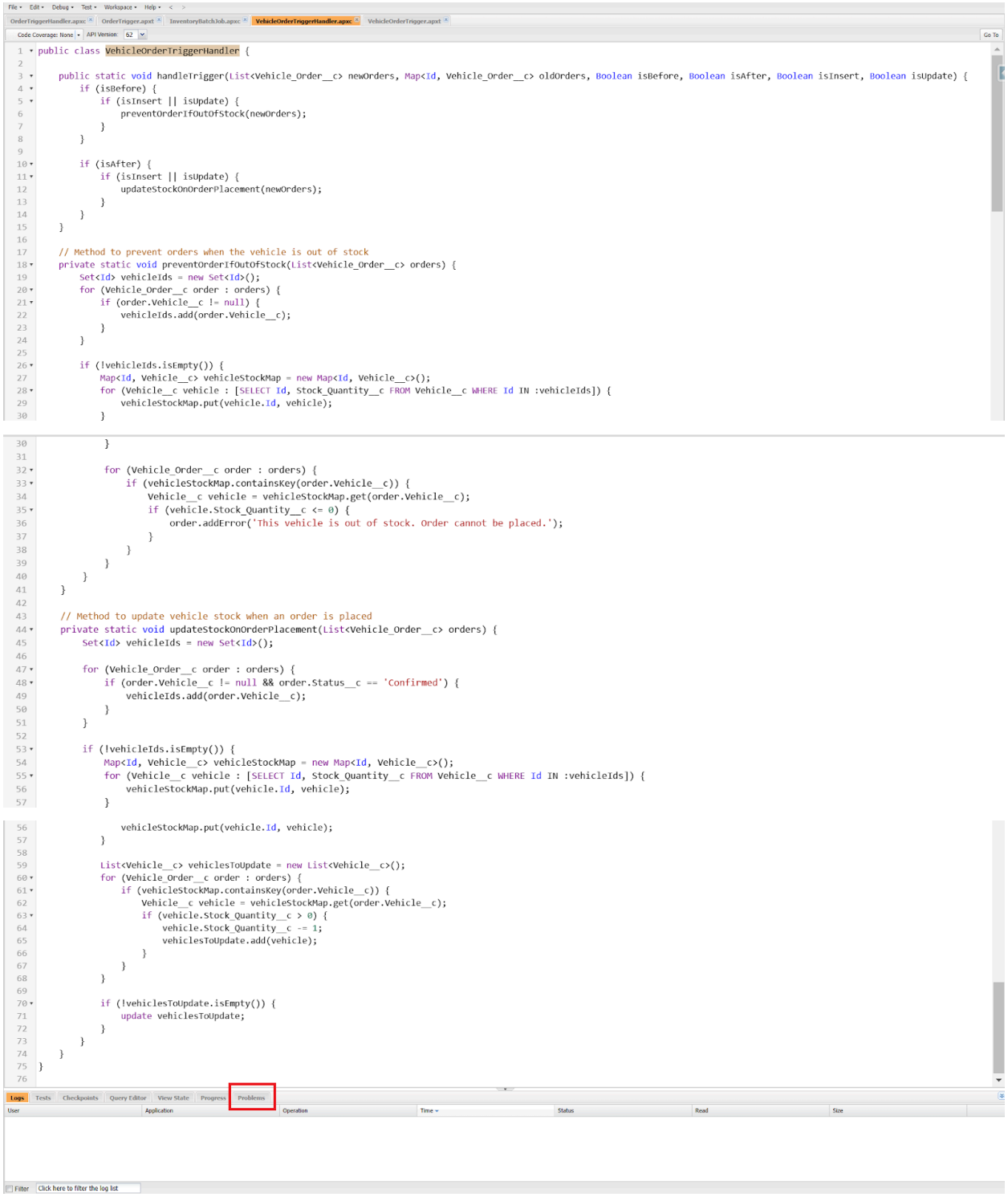


Step 3 : Give Name For Apex Class

- Class Name : VehicleOrderTriggerhandler



Step 4 : Write Apex Code



```
public class VehicleOrderTriggerHandler {

    public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id, Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean isInsert, Boolean isUpdate) {
        if (isBefore) {
            if (isInsert || isUpdate) {
                preventOrderIfOutOfStock(newOrders);
            }
        }

        if (isAfter) {
            if (isInsert || isUpdate) {
                updateStockOnOrderPlacement(newOrders);
            }
        }
    }

    // Method to prevent orders when the vehicle is out of stock
    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }

            for (Vehicle_Order__c order : orders) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c <= 0) {
                        order.addError('This vehicle is out of stock. Order cannot be placed.');
                    }
                }
            }
        }
    }

    // Method to update vehicle stock when an order is placed
    private static void updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();

        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null && order.Status__c == 'Confirmed') {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }

            vehicleStockMap.put(vehicle.Id, vehicle);
        }

        List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
        for (Vehicle_Order__c order : orders) {
            if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle.Stock_Quantity__c > 0) {
                    vehicle.Stock_Quantity__c -= 1;
                    vehiclesToUpdate.add(vehicle);
                }
            }
        }

        if (!vehiclesToUpdate.isEmpty()) {
            update vehiclesToUpdate;
        }
    }
}
```

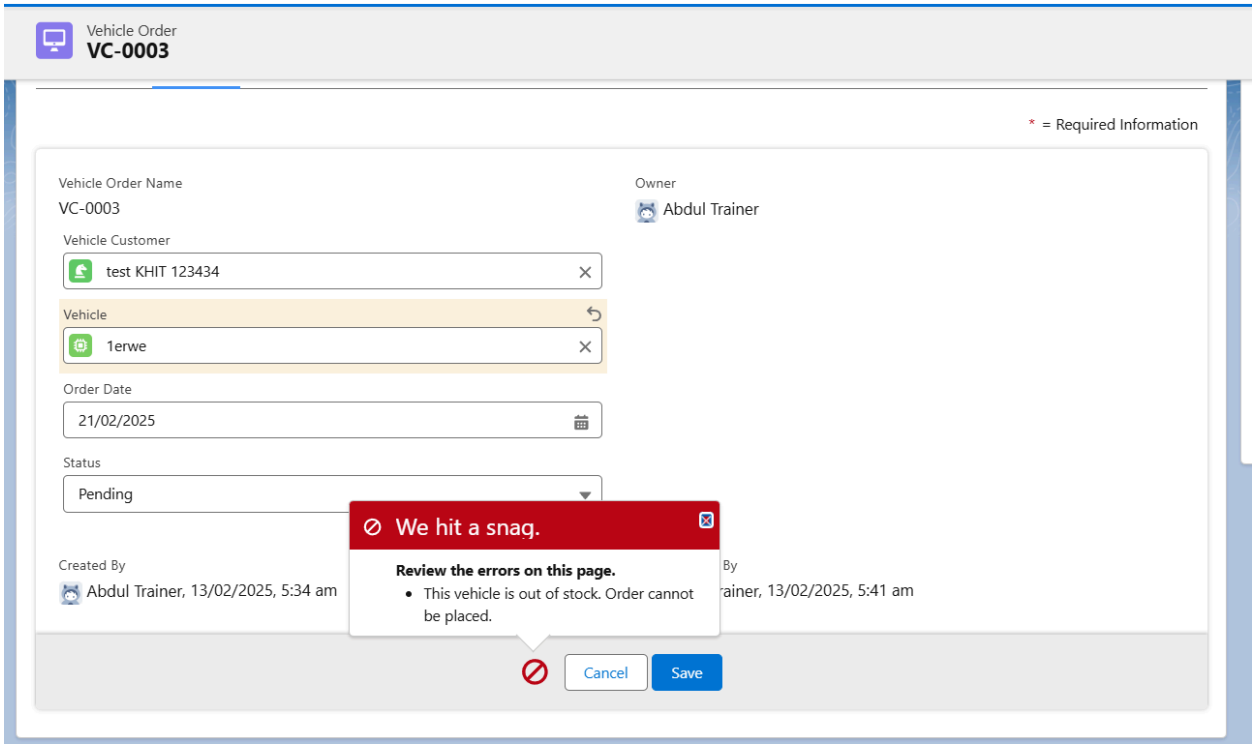
Source Code :

public class VehicleOrderTriggerHandler {

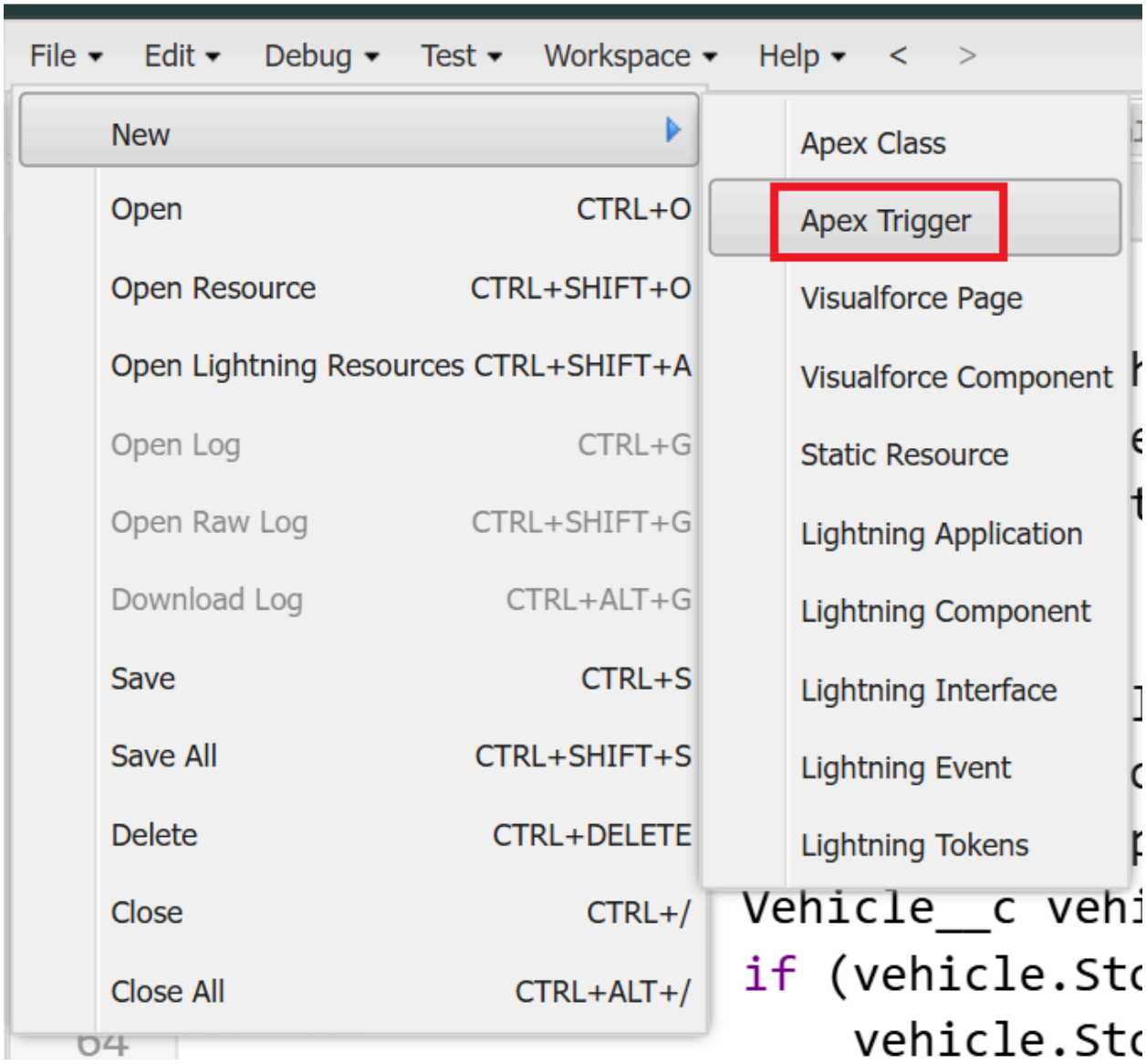
```
    public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id,
Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean isInsert,
Boolean isUpdate) {
        if (isBefore) {
            if (isInsert || isUpdate) {
                preventOrderIfOutOfStock(newOrders);
            }
        }

        if (isAfter) {
            if (isInsert || isUpdate) {
                updateStockOnOrderPlacement(newOrders);
            }
        }
    }
    // Method to prevent orders when the vehicle is out of stock
    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }
```

```
        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c
WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }
            for (Vehicle_Order__c order : orders) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c <= 0) {
                        order.addError('This vehicle is out of stock. Order cannot be placed.');
                    }
                }
            }
        }
    }
    // Method to update vehicle stock when an order is placed
    private static void updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null && order.Status__c == 'Confirmed') {
                vehicleIds.add(order.Vehicle__c);
            }
```

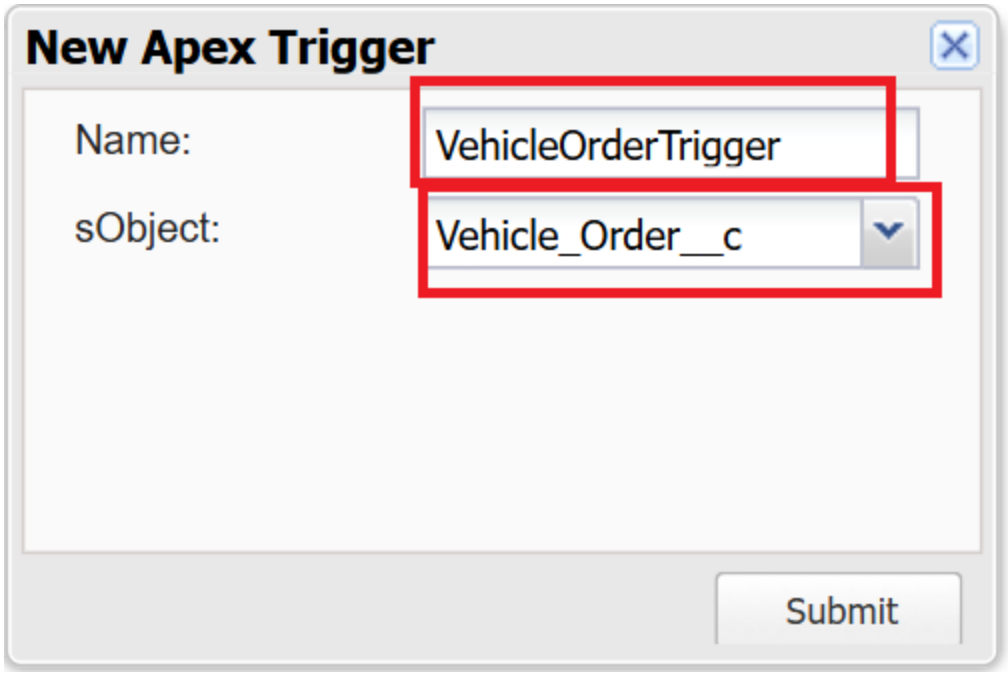
```
        }
        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c
WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }
            List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
            for (Vehicle_Order__c order : orders) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c > 0) {
                        vehicle.Stock_Quantity__c -= 1;
                        vehiclesToUpdate.add(vehicle);
                    }
                }
            }
            if (!vehiclesToUpdate.isEmpty()) {
                update vehiclesToUpdate;
            }
        }
    }
}
```



Step 5 : Write Trigger Handler



Step 6 : Writer Trigger Class Name and Select Vehicle Order Object



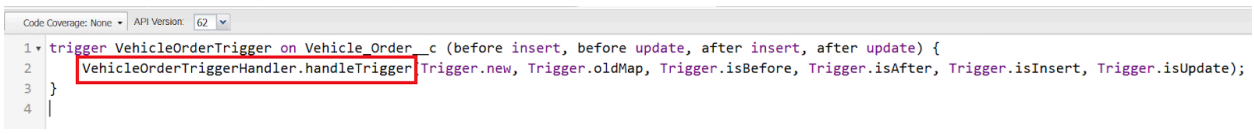
Step 7 : Call Apex Class in Trigger Class

Source Code:

trigger VehicleOrderTrigger on Vehicle_Order__c (before insert,before update, after insert, after update) {

VehicleOrderTriggerHandler.handleTrigger(Trigger.new, Trigger.oldMap, Trigger.isBefore,Trigger.isAfter, Trigger.isInsert, Trigger.isUpdate);

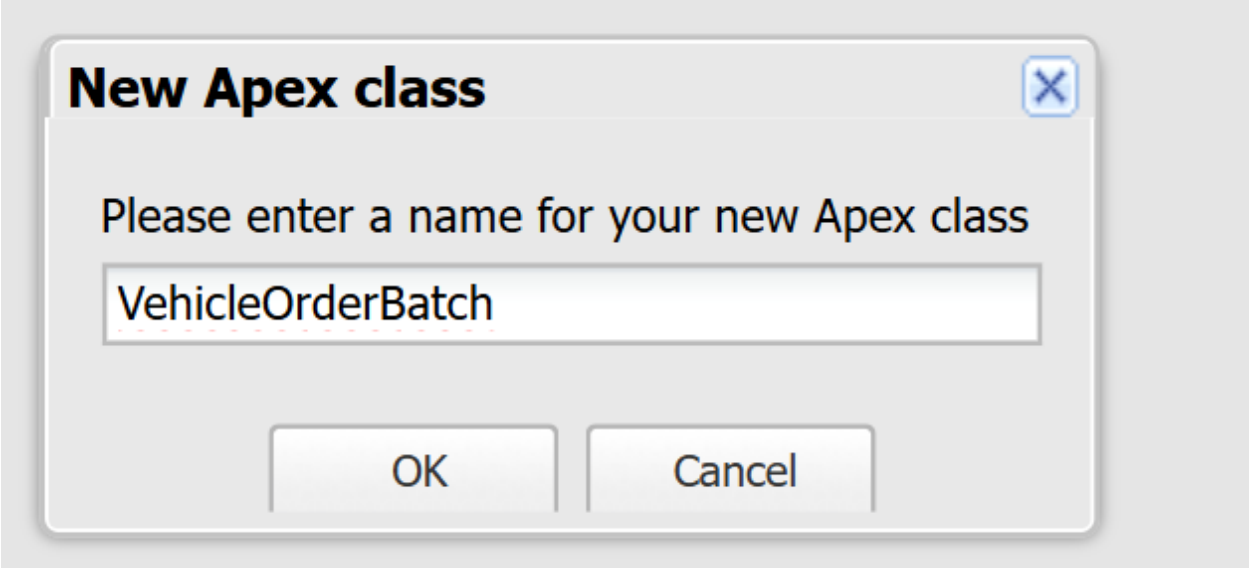
}



Step 8 : Create Batch Job

A customer places an order, but the vehicle is out of stock.

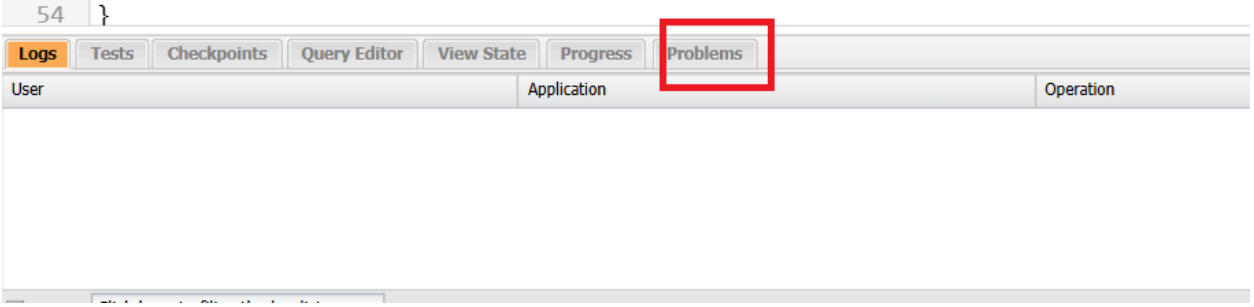
- The order remains pending.
- After new stock is added, the batch job updates the order to confirmed.



Code Coverage: None API Version: 62

```apex
global class VehicleOrderBatch implements Database.Batchable<sObject> {

    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Status__c, Vehicle__c
            FROM Vehicle_Order__c
            WHERE Status__c = 'Pending'
        ]);
    }

    global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
        Set<Id> vehicleIds = new Set<Id>();

        for (Vehicle_Order__c order : orderList) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }

            List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
            List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();

            for (Vehicle_Order__c order : orderList) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
```

```
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c > 0) {
                        order.Status__c = 'Confirmed';
                        vehicle.Stock_Quantity__c -= 1;
                        ordersToUpdate.add(order);
                        vehiclesToUpdate.add(vehicle);
                    }
                }
            }

            if (!ordersToUpdate.isEmpty()) {
                update ordersToUpdate;
            }

            if (!vehiclesToUpdate.isEmpty()) {
                update vehiclesToUpdate;
            }
        }
    }

    global void finish(Database.BatchableContext bc) {
        System.debug('Vehicle order batch job completed.');
    }
}
```



Source Code:

global class VehicleOrderBatch implements Database.Batchable<sObject> {

global Database.QueryLocator start (Database.BatchableContext bc) {

return Database.getQueryLocator([

SELECT Id, Status_c, Vehicle___c

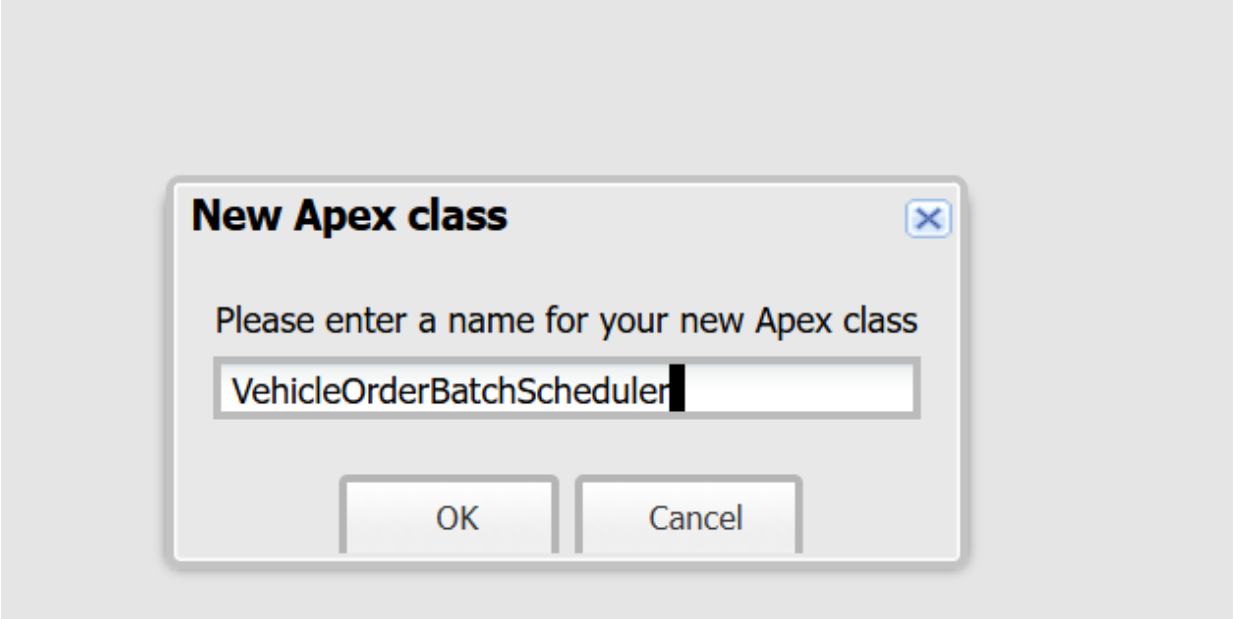
FROM Vehicle_Order___c

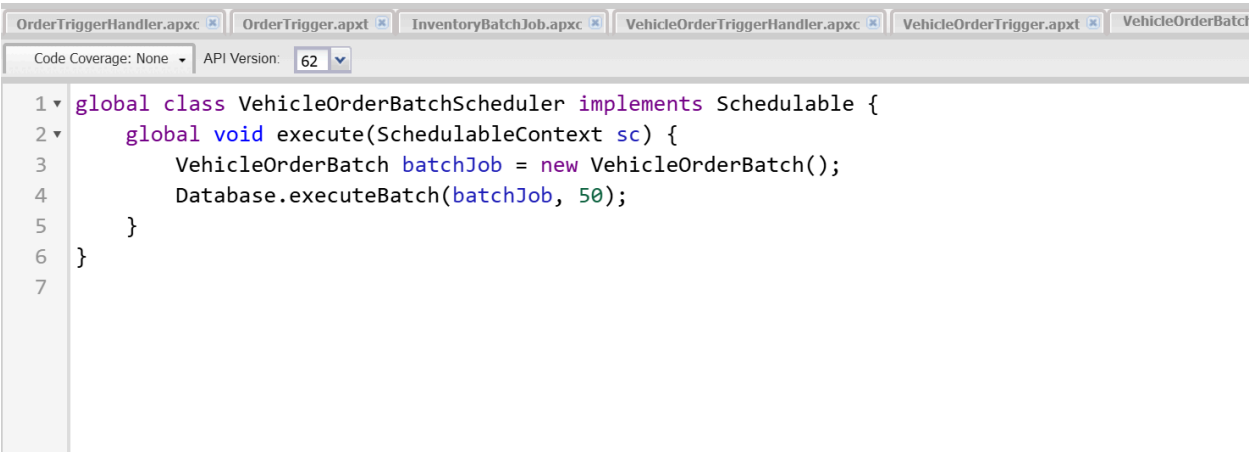
WHERE Status__c

```
'Pending'

]);

}

global void execute(Database.BatchableContext bc, List<Vehicle_Order_c> orderList) {

Set<Id> vehicleIds = new Set<Id>();

for (Vehicle_Order_c order orderList) {

if (order.Vehicle_c != null) {

vehicleIds.add(order.Vehicle__c);

}

}

if (!vehicleIds.isEmpty()) {

Map<Id, Vehicle___c> vehicleStockMap = new Map<Id, Vehicle___c>();

for (Vehicle_c vehicle: [SELECT Id, Stock Quantity_c FROM Vehicle_c WHERE Id IN
vehicleIds]) {

vehicleStockMap.put(vehicle. Id, vehicle);

}

List<Vehicle_Order_c> ordersToUpdate = new List<Vehicle_Order_c>();
List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();

for (Vehicle_Order_c order orderList) {

if (vehicleStockMap.containsKey(order. Vehicle__c)) {

Vehicle_c vehicle = vehicleStockMap.get(order. Vehicle__c);

if (vehicle.Stock_Quantity___c > 0) {

order.Status___c = 'Confirmed';
```

```
vehicle.Stock_Quantity___c -= 1;
ordersToUpdate.add(order);
vehiclesToUpdate.add(vehicle);
}
}
}
if (!ordersToUpdate.isEmpty()) {
update ordersToUpdate;
}
if (!vehicles ToUpdate.isEmpty()) {
update vehiclesToUpdate;
}
}
global void finish(Database.BatchableContext bc) {
System.debug('Vehicle order batch job completed.');
}
}
```

Step 9 : Create Schedule Class then revoked Batch Class in Schedule Class

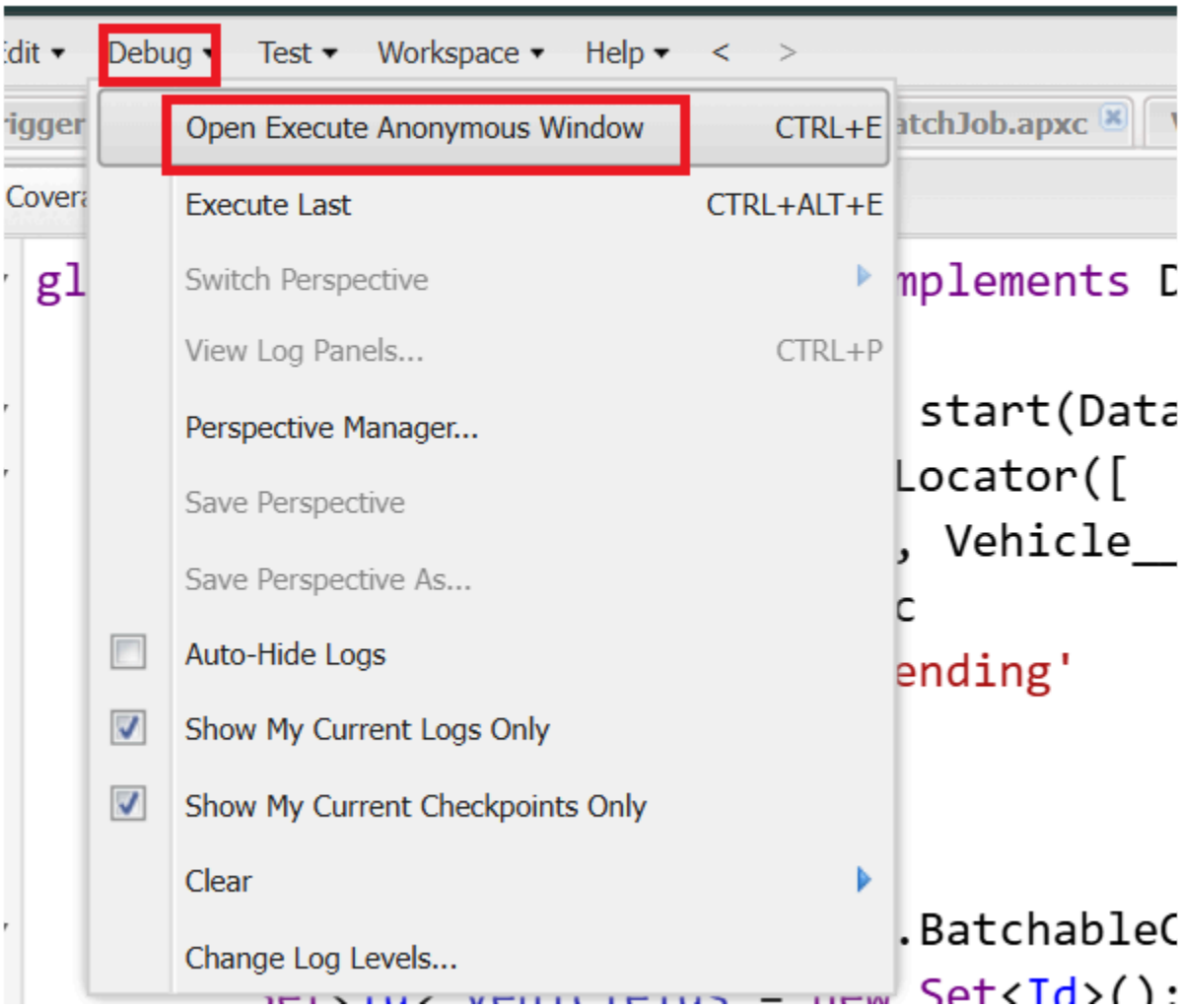




Source Code :

```
global class VehicleOrderBatchScheduler implements Schedulable {
  global void execute(SchedulableContext sc) {
    VehicleOrderBatch batchJob = new VehicleOrderBatch();
    Database.executeBatch(batchJob, 50); // 50 is the batch size
  }
}
```

Step 10 : Schedule the Batch Job To run the batch job **every night at midnight** :

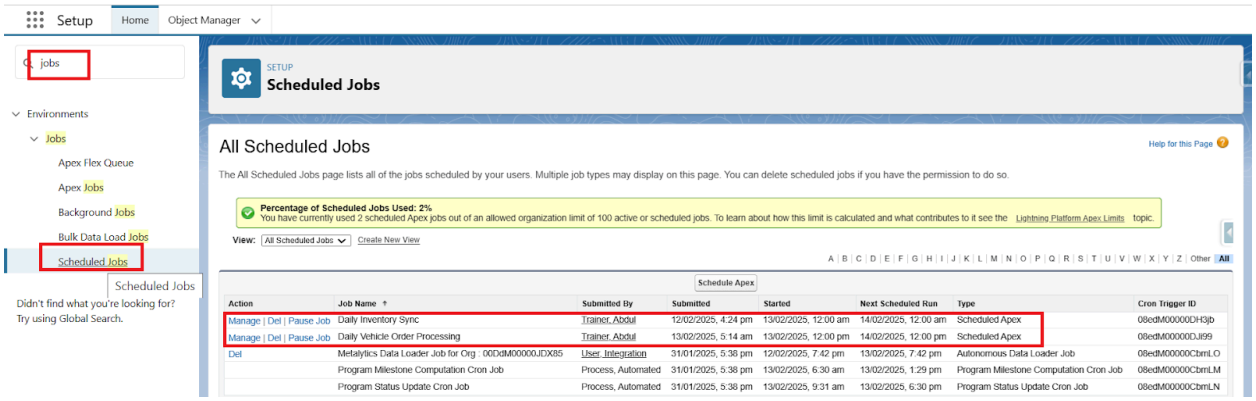


Source Code :

String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM

System.schedule('Daily Vehicle Order Processing', cronExp, new VehicleOrderBatchScheduler());

Step 11 : You Can Check where this Schedule Job is Running



7. FUNCTIONAL & PERFORMANCE TESTING

During testing, screenshots were captured from Salesforce to validate vehicle creation, dealer assignment, stock validation triggers, order status updates, batch processing, scheduled reminders, and dashboard analytics. All features were verified to ensure correct functionality and performance.

8. ADVANTAGES & DISADVANTAGES

Advantages

- Centralized data
- Automated workflows
- Real-time reporting

Disadvantages

- Salesforce dependency
- Requires initial configuration effort

9. Conclusion:

The Salesforce CRM implementation at WhatNext Vision Motors successfully modernizes vehicle ordering and dealership management. Automation improves efficiency, accuracy, and customer experience.

10. FUTURE SCOPE

- Mobile ordering app
- Dealer GPS integration
- SMS notifications
- AI-based demand forecasting

11. APPENDIX

- Source Code: Salesforce Configuration
- Dataset: Salesforce Objects
- GitHub / Demo: Salesforce Developer Org

Thank You